CS 422 Lab 2 Writeup

Implementation Note: automatically queries iperf3.org and generates a random list of targets, retrying when targets are unavailable.

```
> cat Dockerfile
FROM ubuntu:26.04

ENV DEBIAN_FRONTEND=noninteractive

# note: run sudo modprobe tcp_bbr on host machine to enable bbr
RUN apt update && apt install -y \
    curl \
    vim \
    gcc \
    build-essential \
    python3 \
    python3-pip \
    python3-requests \
    python3-numpy \
    python3-matplotlib

CMD ["bash"]

> python3 main.py --help
usage: python3 main.py {-f</path/to/server_list_file:str> |
-n<#destinations:int=10>} <duration:int=60>
<interval:float=0.5> <ccalg:str=cubic> <repetitions:int=1>

> python3 main.py 10 5 0.5 cubic
summaries for cubic:
                   host             minimum
median                mean    95-th percentile
speedtest.mtl2.ca.leasew     197.072 Mbits/s     539.416 Mbits/
s     512.570 Mbits/s     579.324 Mbits/s
             185.93.1.65     103.590 Mbits/s     194.918 Mbits/
s     184.269 Mbits/s     240.677 Mbits/s
 speed-cb.dimensione.com       1.480 Mbits/s     148.426 Mbits/
s     108.879 Mbits/s     154.323 Mbits/s
           138.199.14.66       3.317 Mbits/s      89.311 Mbits/
s      93.518 Mbits/s     171.691 Mbits/s
           156.146.40.65       2.089 Mbits/s     166.564 Mbits/
s     136.931 Mbits/s     176.539 Mbits/s
 yyc-speedtest.xplore.ca      11.549 Mbits/s     280.324 Mbits/
```

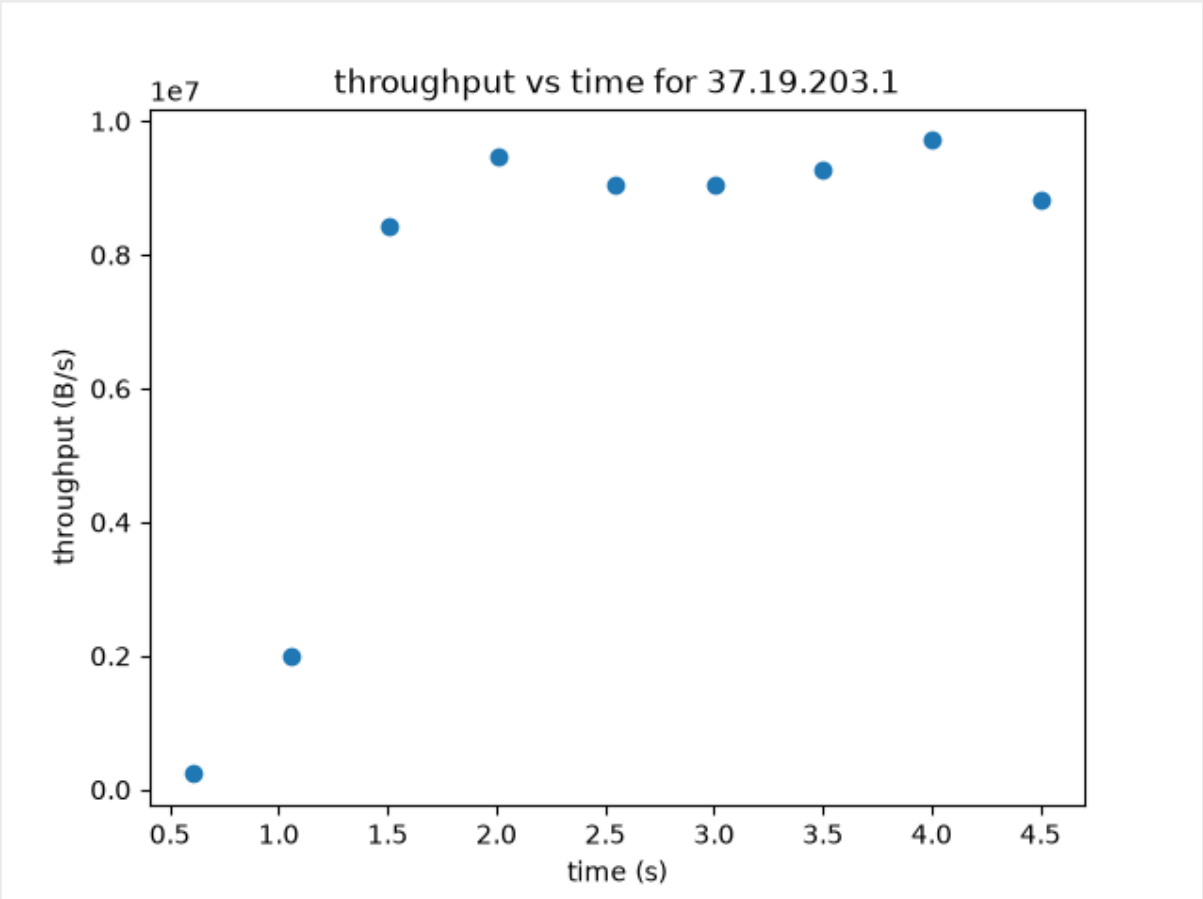
```
s     241.349 Mbits/s     286.448 Mbits/s
ams.speedtest.clouvider.       3.923 Mbits/s     210.899 Mbits/
s     174.632 Mbits/s     214.295 Mbits/s
            185.3.160.57       3.489 Mbits/s     194.747 Mbits/
s     155.343 Mbits/s     197.875 Mbits/s
     speedtest.nocix.net     164.761 Mbits/s     321.511 Mbits/
s     307.158 Mbits/s     337.176 Mbits/s
            109.61.94.65       1.937 Mbits/s     162.002 Mbits/
s     126.704 Mbits/s     168.592 Mbits/s

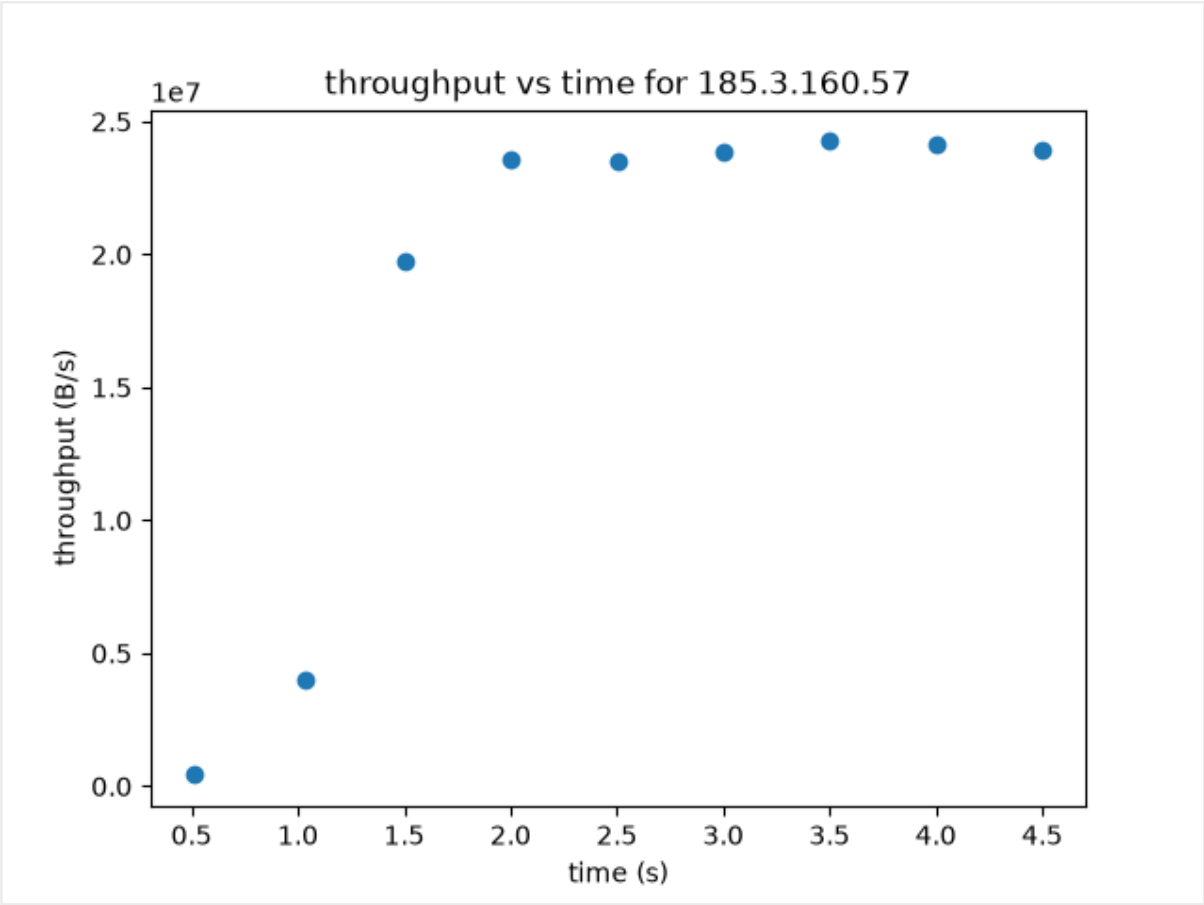
> python3 main.py 10 5 0.5 bbr
summaries for bbr:
                    host             minimum
median                mean    95-th percentile
dal.speedtest.clouvider.     250.688 Mbits/s     364.599 Mbits/
s     420.199 Mbits/s     586.963 Mbits/s
       syd.proof.ovh.net     681.860 Kbits/s     104.052 Mbits/
s      69.018 Mbits/s     108.451 Mbits/s
la.speedtest.clouvider.n      35.787 Mbits/s     349.542 Mbits/
s     313.500 Mbits/s     358.933 Mbits/s
            37.19.218.65       1.289 Mbits/s     146.186 Mbits/
s     111.427 Mbits/s     153.890 Mbits/s
          105.235.237.2     352.664 Kbits/s      64.590 Mbits/
s      40.885 Mbits/s      67.187 Mbits/s
            84.17.57.129     396.683 Kbits/s      65.621 Mbits/
s      46.933 Mbits/s      87.004 Mbits/s
atl.speedtest.clouvider.     290.458 Mbits/s     516.642 Mbits/
s     485.626 Mbits/s     654.831 Mbits/s
         185.246.208.67       1.556 Mbits/s     163.182 Mbits/
s     125.773 Mbits/s     165.439 Mbits/s
   spd-icsrv.hostkey.com       1.320 Mbits/s     156.858 Mbits/
s     121.303 Mbits/s     161.254 Mbits/s
            185.3.160.57       2.565 Mbits/s     194.661 Mbits/
s     158.215 Mbits/s     198.332 Mbits/s

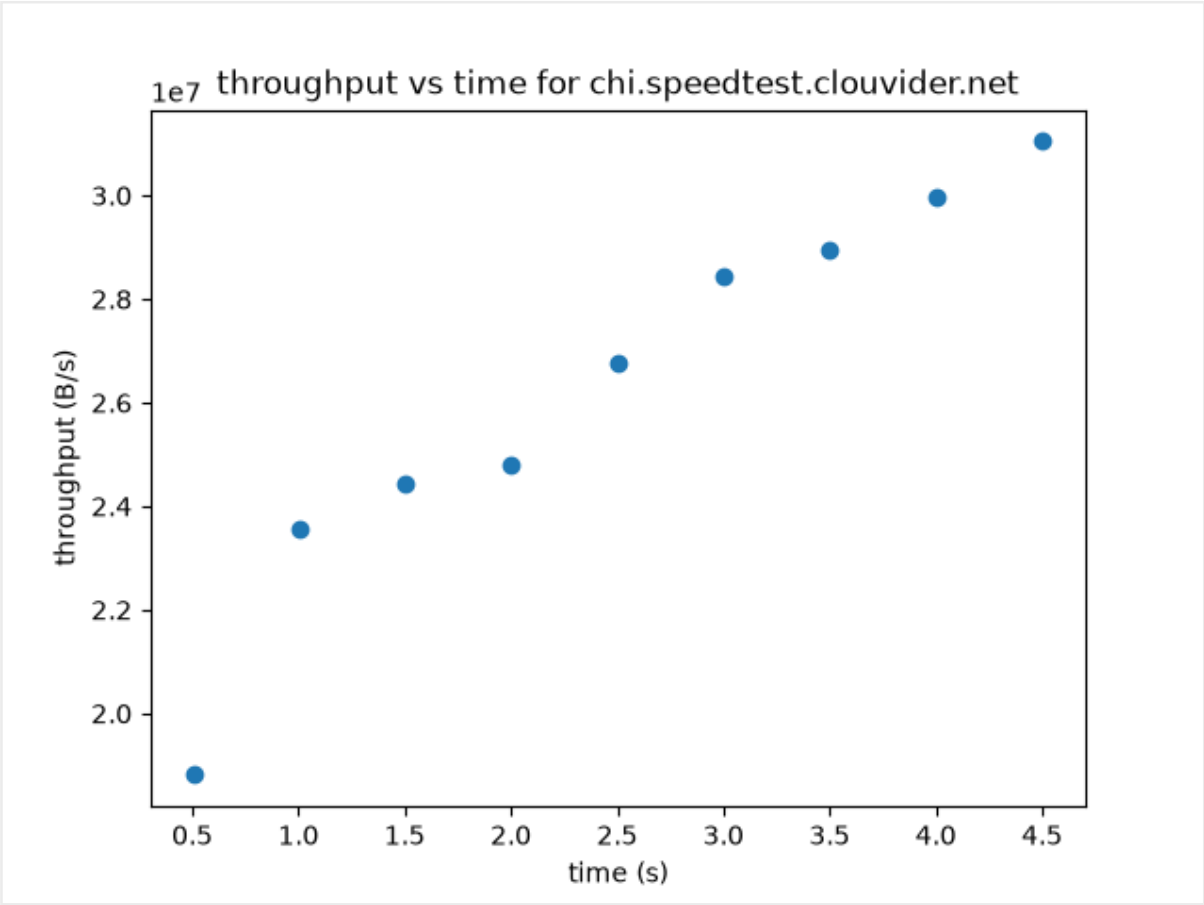
> python3 main.py 10 5 0.5 reno
summaries for reno:
                    host             minimum
median                mean    95-th percentile
chi.speedtest.clouvider.     344.308 Mbits/s     499.555 Mbits/
s     486.289 Mbits/s     524.849 Mbits/s
speedtest.phx1.us.leasew      53.222 Mbits/s     374.166 Mbits/
s     341.490 Mbits/s     386.936 Mbits/s
speedtest.sfo12.us.lease      45.316 Mbits/s     354.344 Mbits/
```

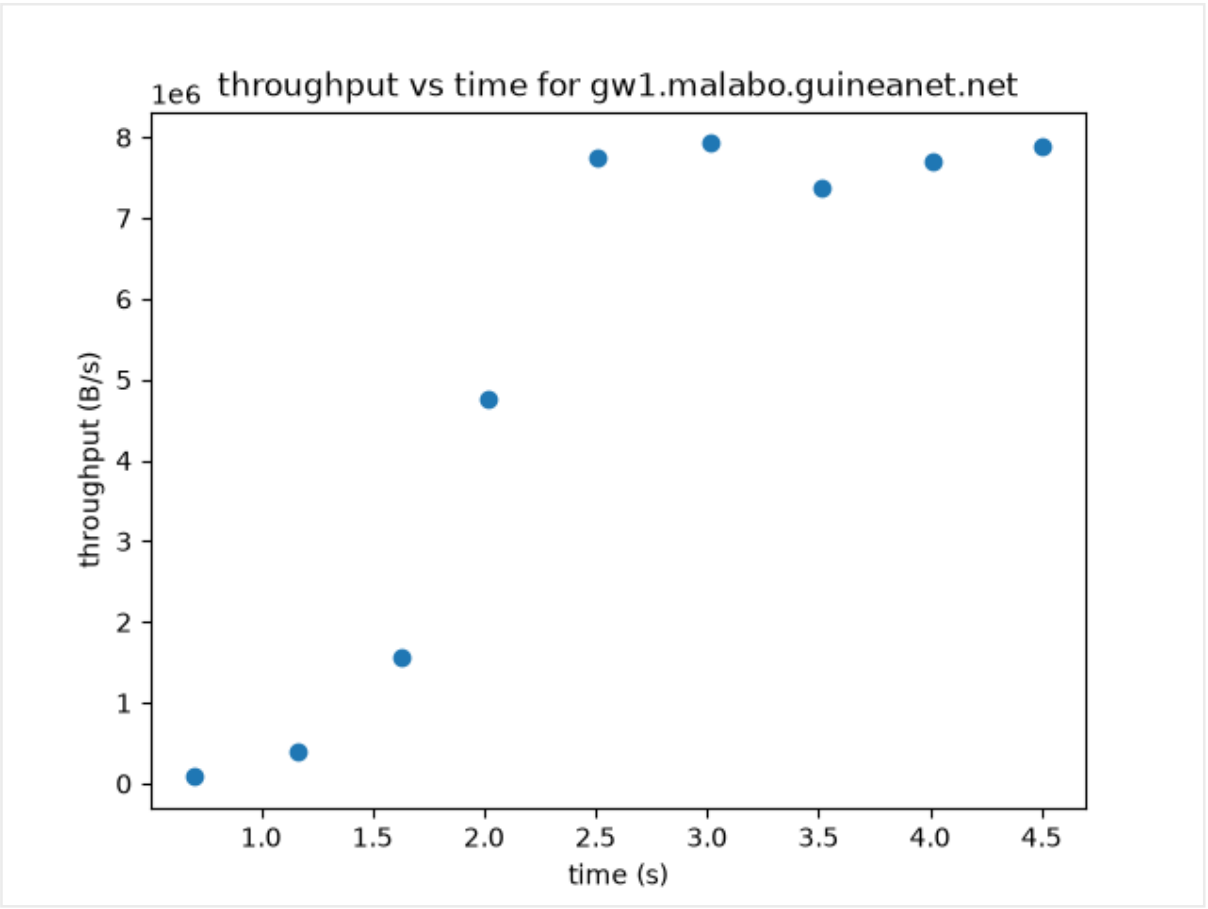
```
s     323.367 Mbits/s     374.799 Mbits/s
a209.speedtest.wobcom.de       2.167 Mbits/s     166.179 Mbits/
s     133.205 Mbits/s     171.823 Mbits/s
 speedtest.shinternet.ch       3.465 Mbits/s     180.652 Mbits/
s     142.548 Mbits/s     184.956 Mbits/s
   spd-icsrv.hostkey.com       2.089 Mbits/s     157.981 Mbits/
s     124.781 Mbits/s     166.960 Mbits/s
       syd.proof.ovh.net     997.249 Kbits/s     102.121 Mbits/
s      70.678 Mbits/s     114.347 Mbits/s
a204.speedtest.wobcom.de       1.648 Mbits/s     161.761 Mbits/
s     125.652 Mbits/s     167.784 Mbits/s
           138.199.14.66       2.657 Mbits/s     178.189 Mbits/
s     140.528 Mbits/s     179.025 Mbits/s
  bwtest.linxtelecom.com       1.616 Mbits/s     139.215 Mbits/
s     107.306 Mbits/s     145.815 Mbits/s
```

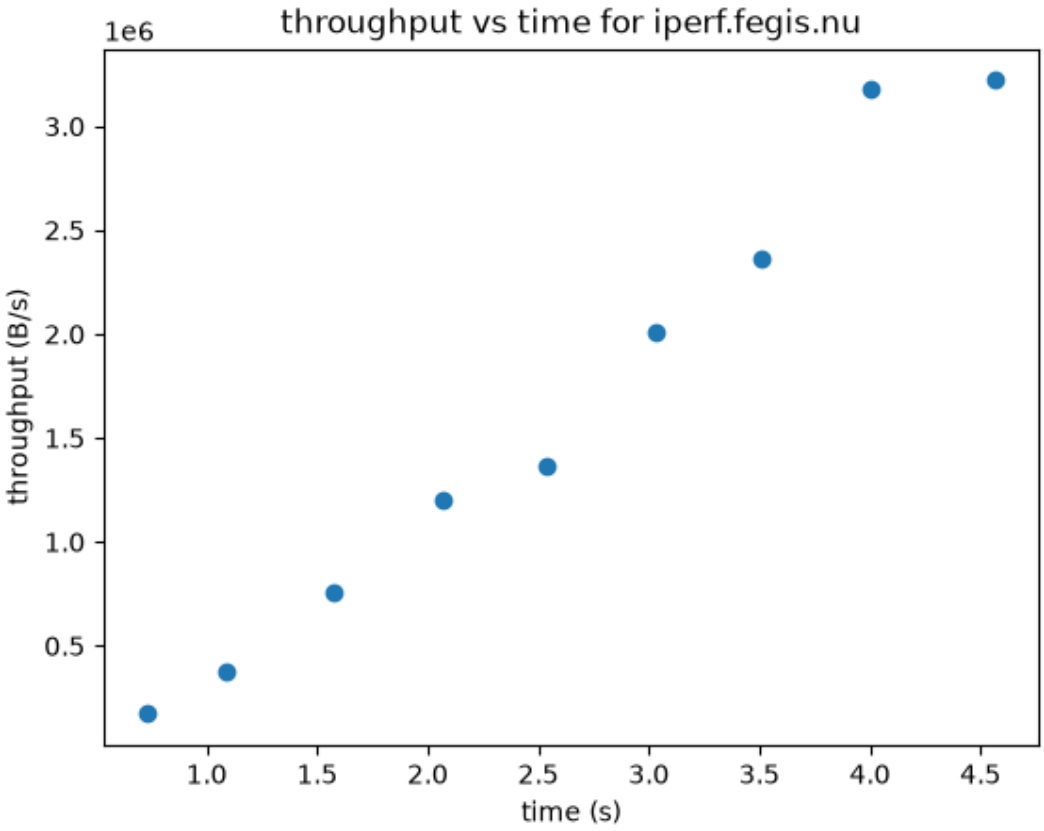
Part 1: Throughput vs Time

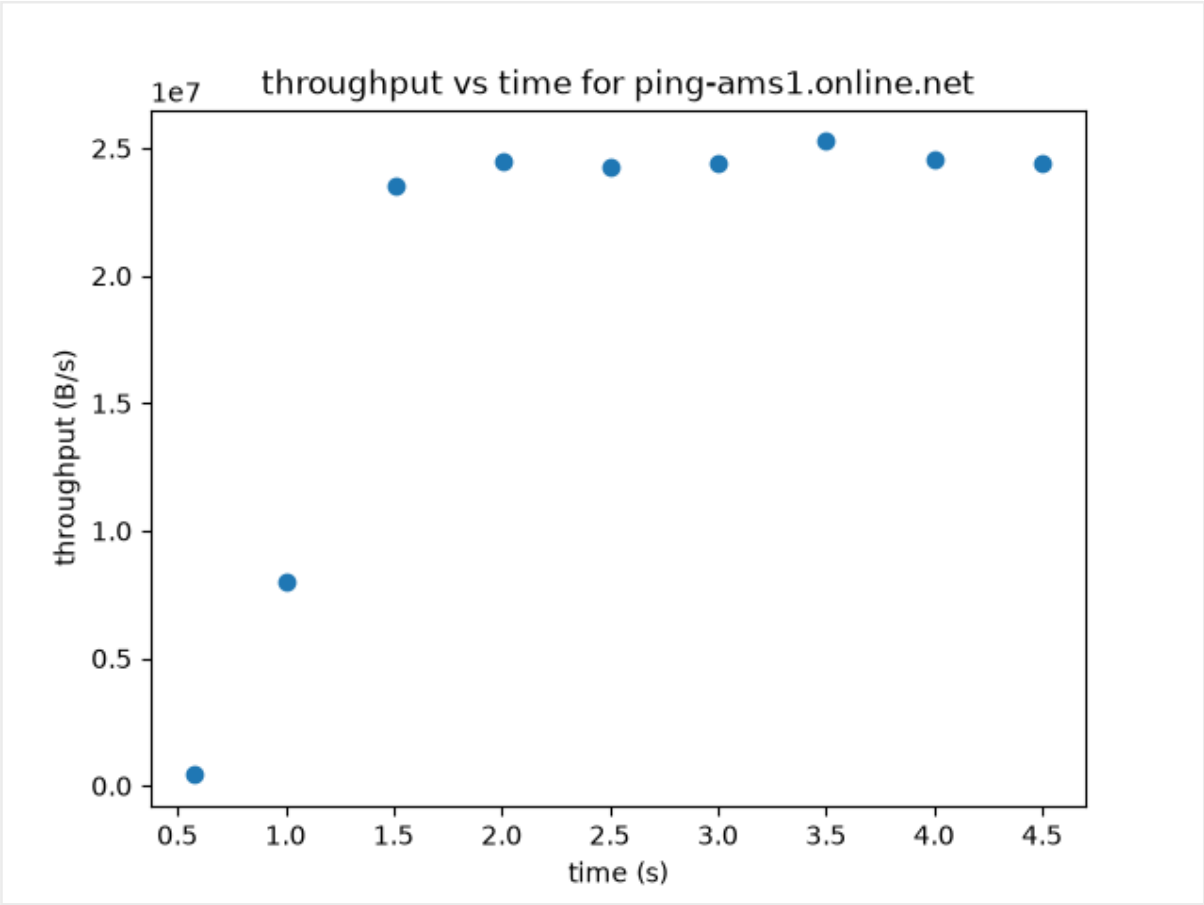


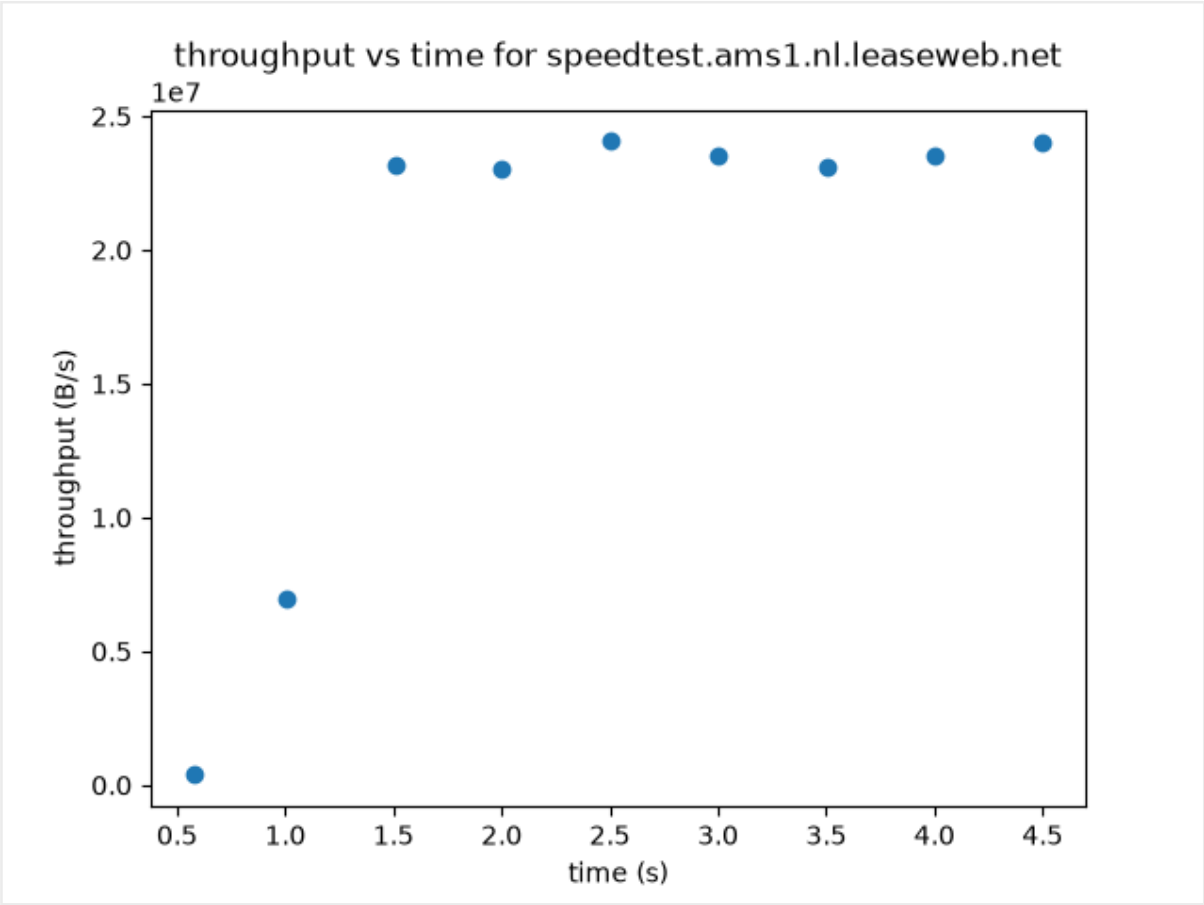

throughput vs time for 185.3.160.57
1e7
2.5
2.0
1.5
1.0
0.5
0.0
throughput (B/s)
0.5
1.0
1.5
2.0
2.5
3.0
3.5
4.0
4.5
time (s)

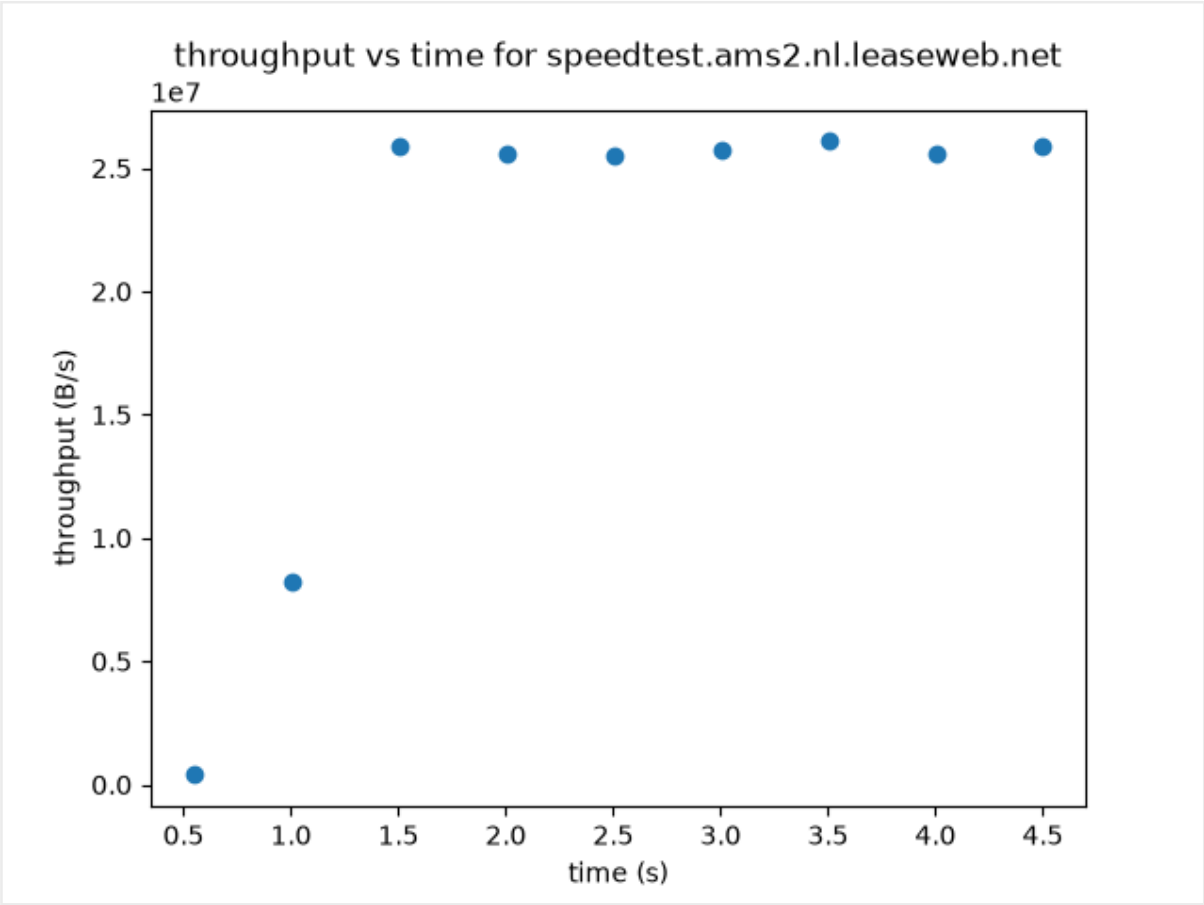

throughput vs time for chi.speedtest.clouvider.net
1e7
3.0
2.8
2.6
2.4
2.2
2.0
throughput (B/s)
0.5
1.0
1.5
2.0
2.5
3.0
3.5
4.0
4.5
time (s)

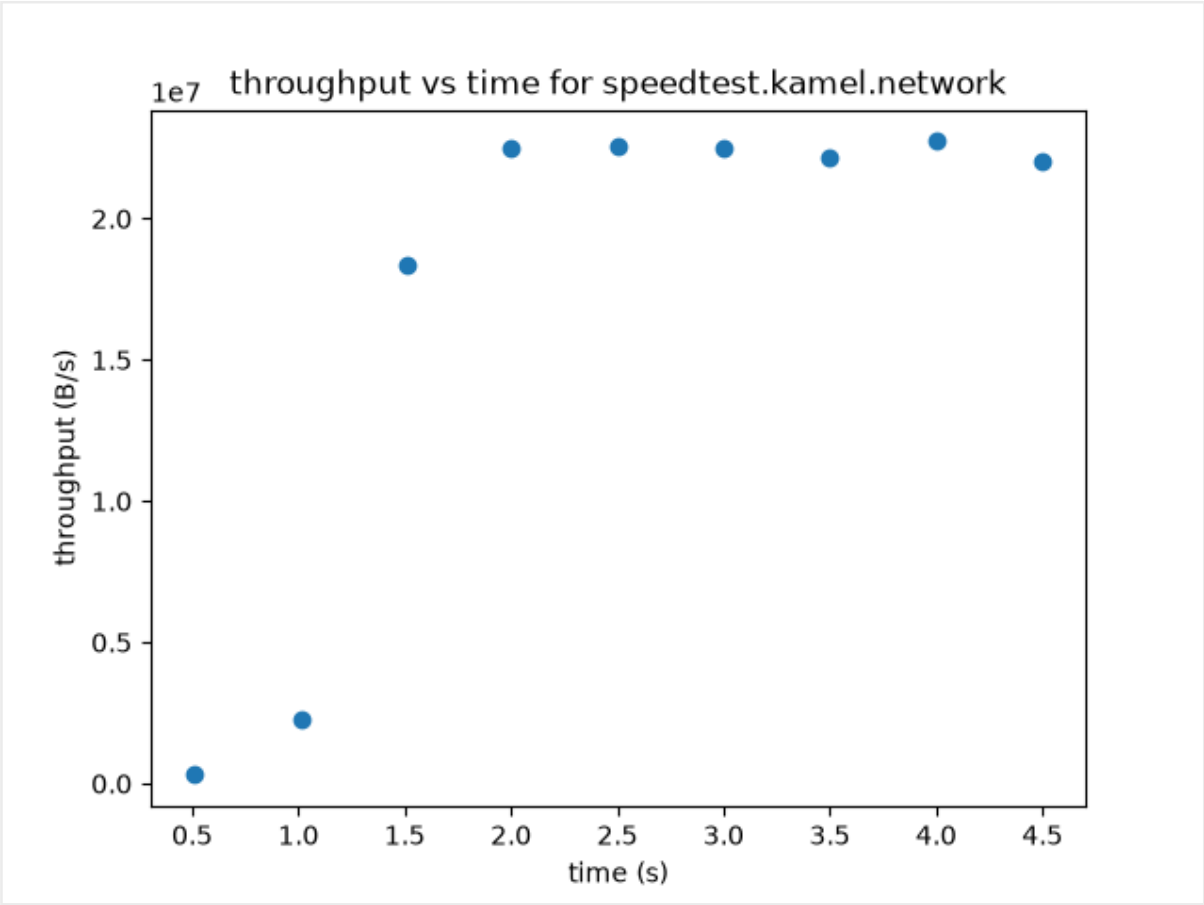

throughput vs time for gw1.malabo.guineanet.net
1e6
8
7
6
5
4
3
2
1
0
throughput (B/s)
1.0
1.5
2.0
2.5
3.0
3.5
4.0
4.5
time (s)

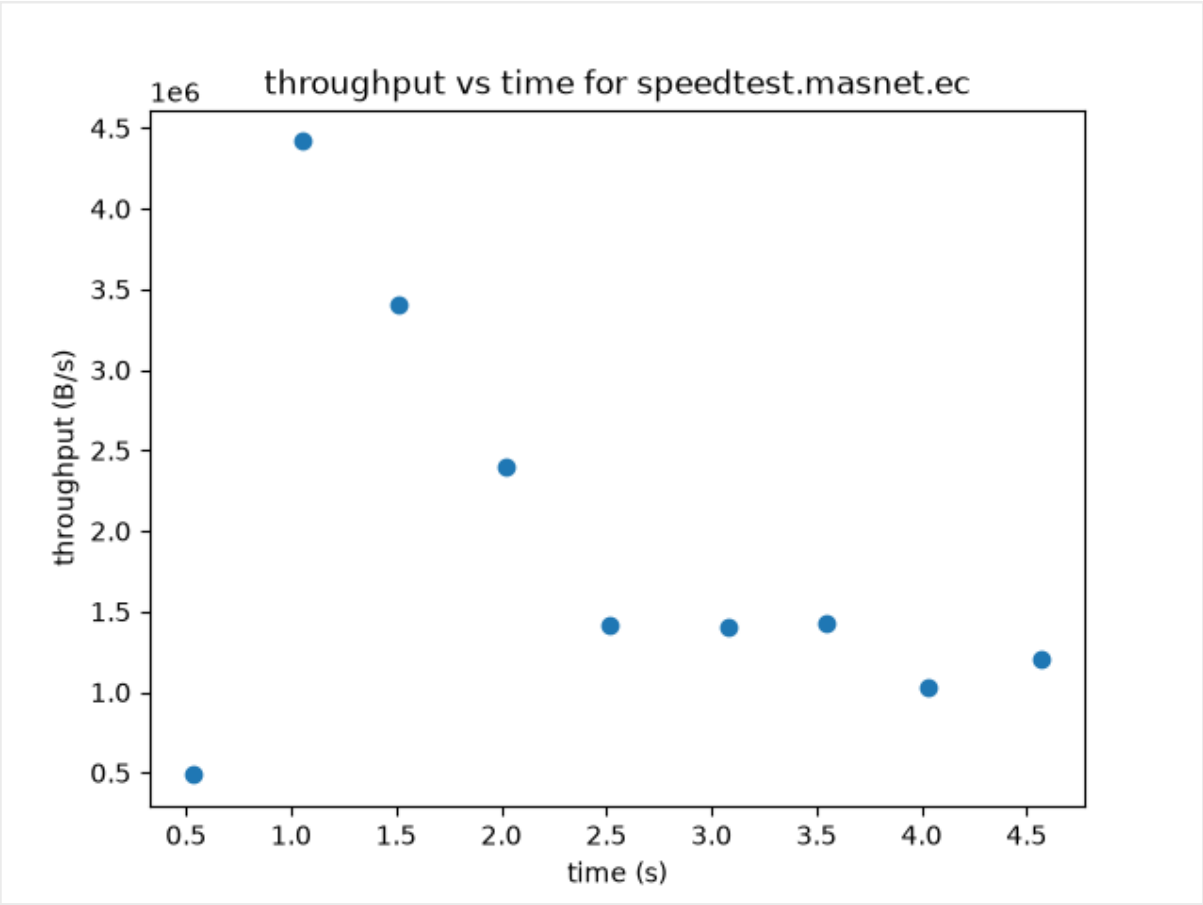

throughput vs time for iperf.fegis.nu
1e6
3.0
2.5
2.0
1.5
1.0
0.5
throughput (B/s)
1.0
1.5
2.0
2.5
3.0
3.5
4.0
4.5
time (s)


throughput vs time for ping-ams1.online.net
1e7
2.5
2.0
1.5
1.0
0.5
0.0
throughput (B/s)
0.5
1.0
1.5
2.0
2.5
3.0
3.5
4.0
4.5
time (s)

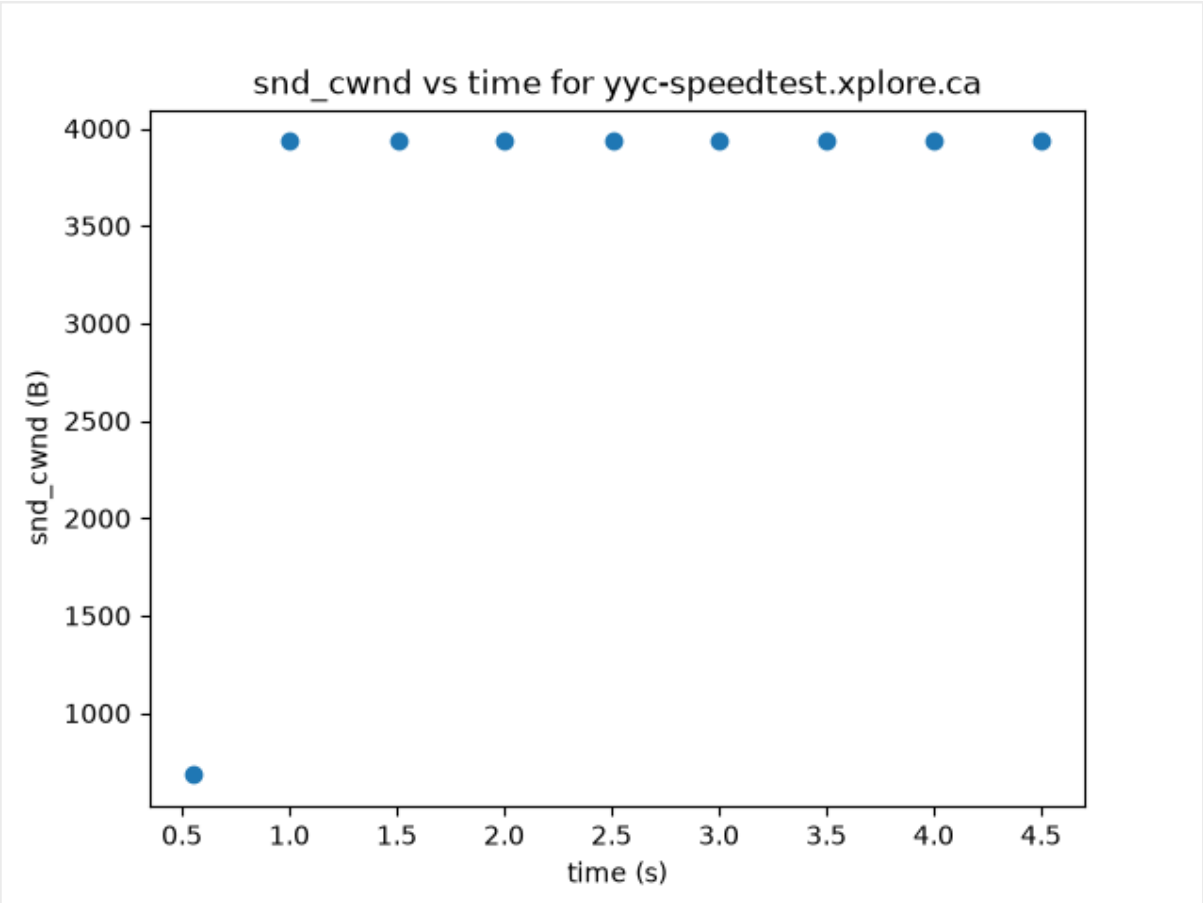

throughput vs time for speedtest.ams1.nl.leaseweb.net
1e7
2.5
2.0
1.5
1.0
0.5
0.0
throughput (B/s)
0.5
1.0
1.5
2.0
2.5
3.0
3.5
4.0
4.5
time (s)


throughput vs time for speedtest.ams2.nl.leaseweb.net
1e7
2.5
2.0
1.5
1.0
0.5
0.0
throughput (B/s)
0.5
1.0
1.5
2.0
2.5
3.0
3.5
4.0
4.5
time (s)

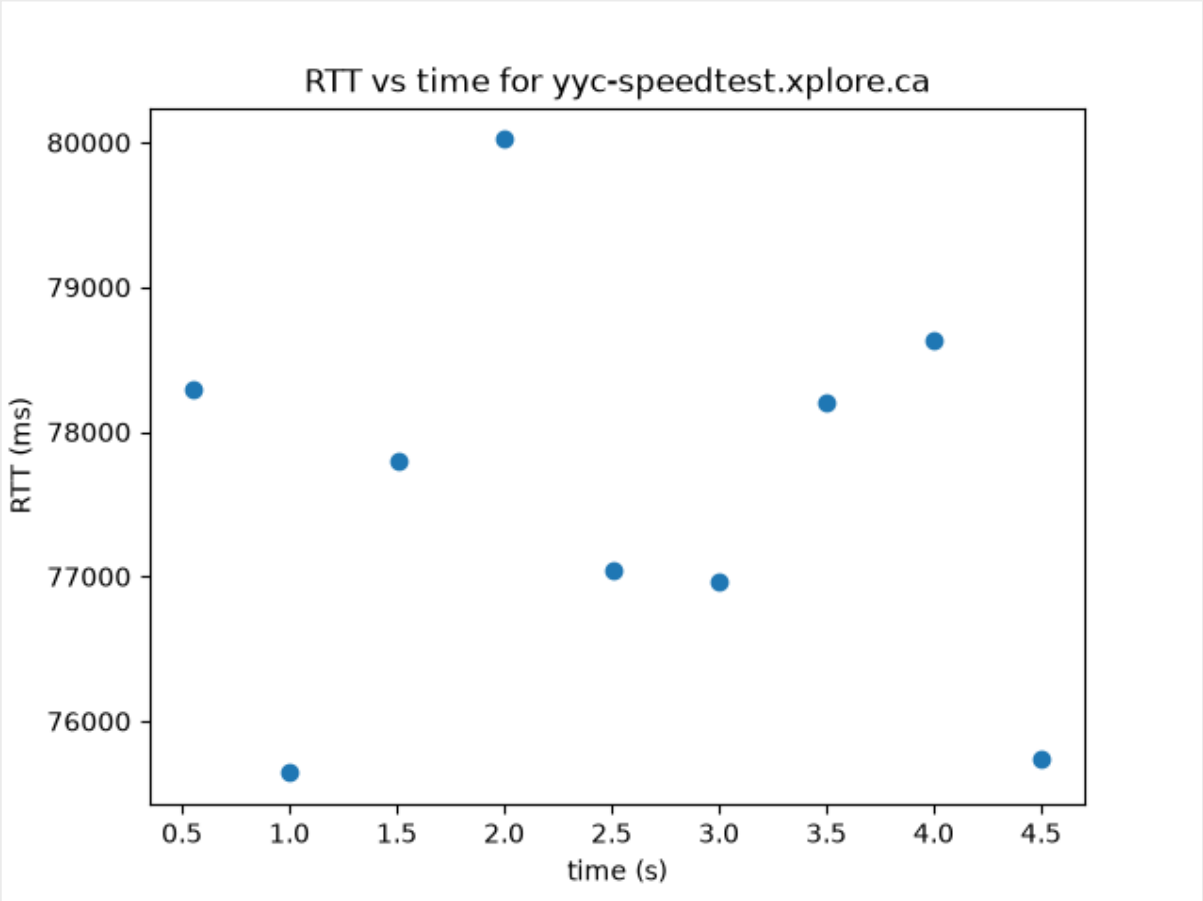

throughput vs time for speedtest.kamel.network
1e7
throughput (B/s)
0.0
0.5
1.0
1.5
2.0
0.5
1.0
1.5
2.0
2.5
3.0
3.5
4.0
4.5
time (s)



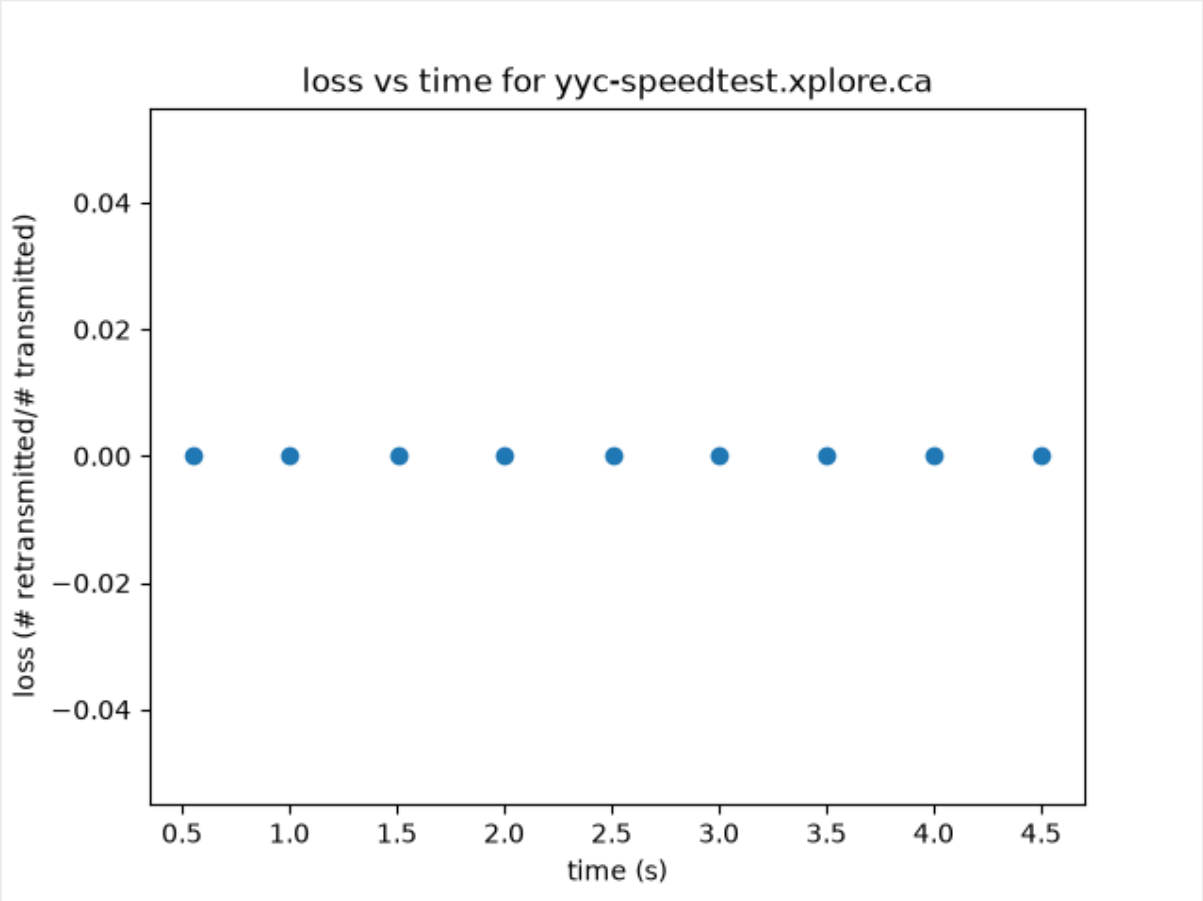
Part 2: CUBIC TCP Statistics for yyc-speedtest.xplore.ca



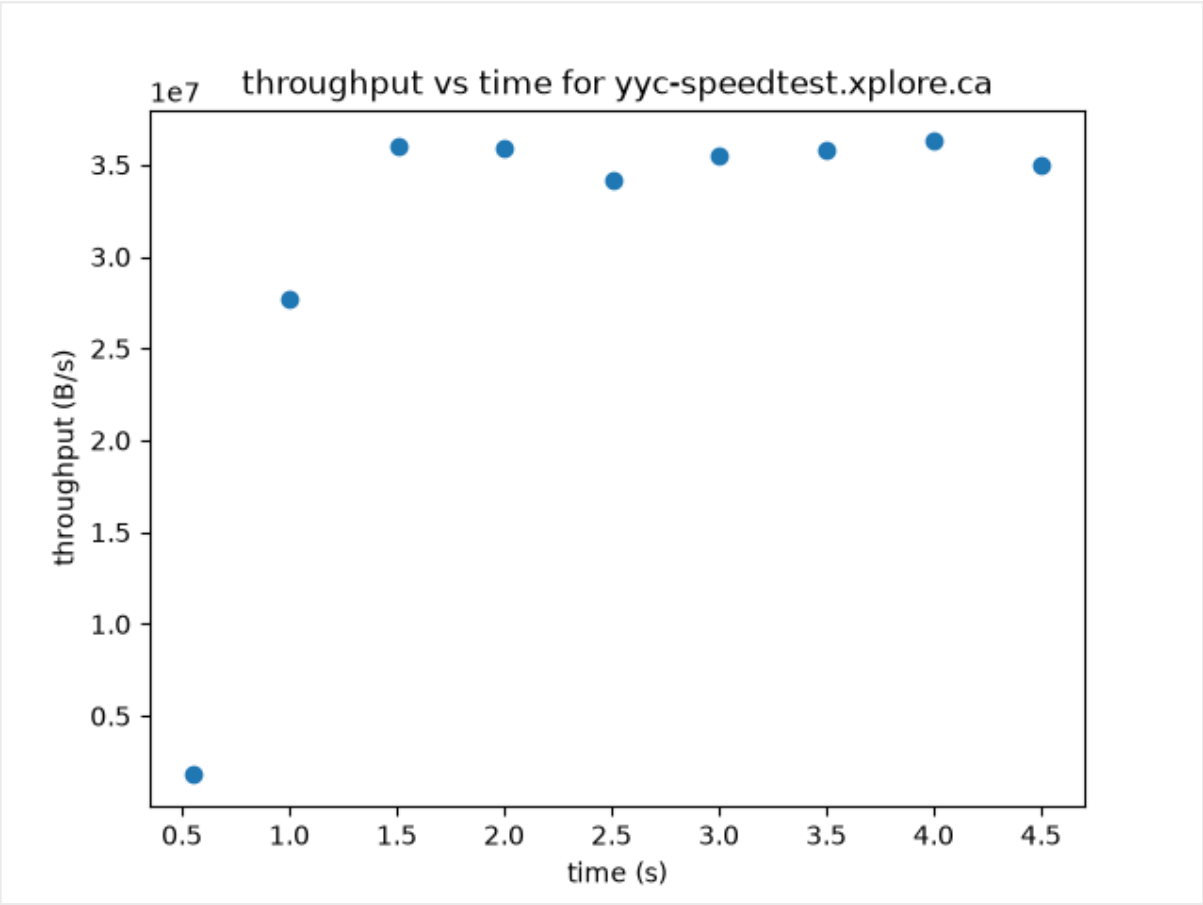
- snd_cwnd denotes the send congestion window, this graph shows that it starts small, then jumps up to a high point and stays there, with a faster sample size, the cubic function would be visible, quickly ramping it to the known good value and holding there with low deviation. goodput likely looks similar to this graph, higher sending windows likely means higher good throughput, provided the sending window was calculated (and the stability indicates it was)



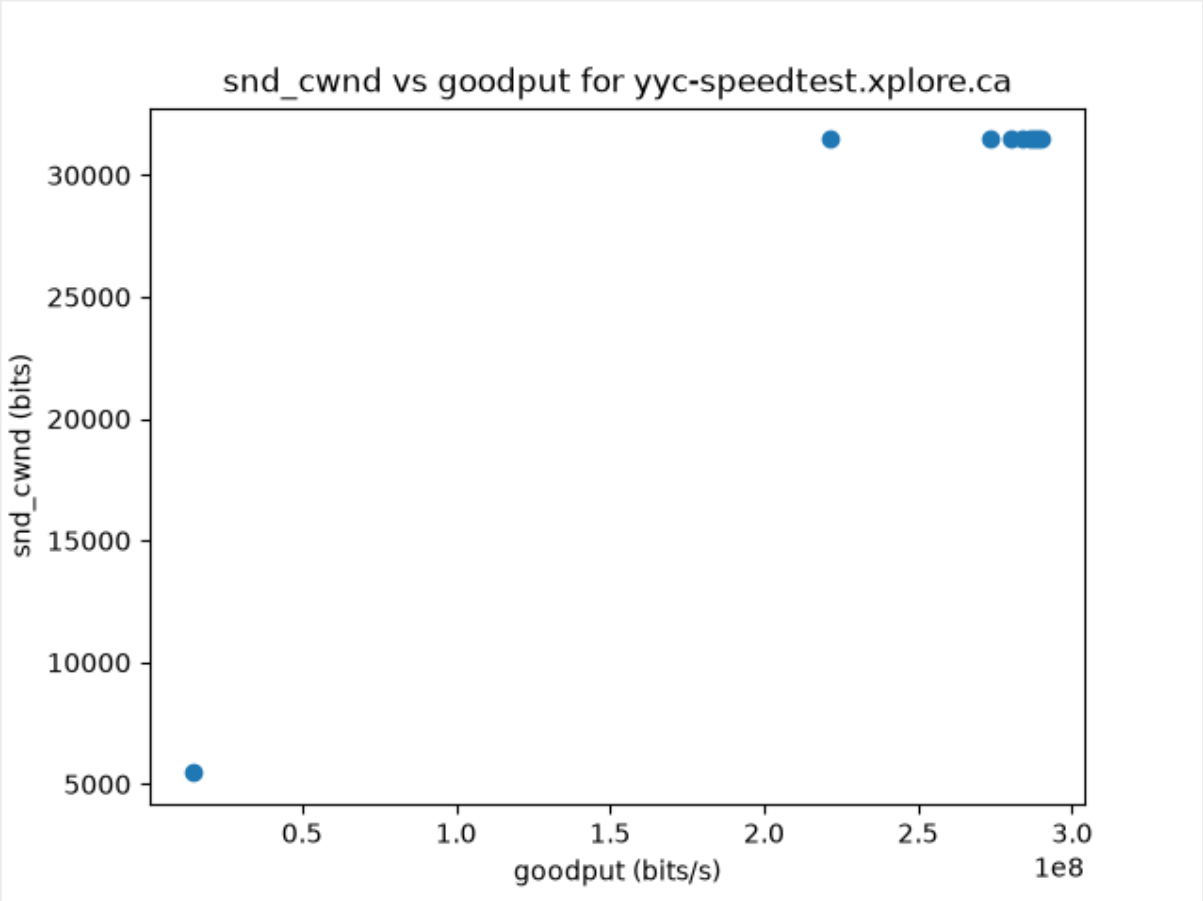
- RTT denotes the instantaneous average round trip time for outgoing packets and is all over the place depending on what is going on in the network, this graph shows no clear pattern



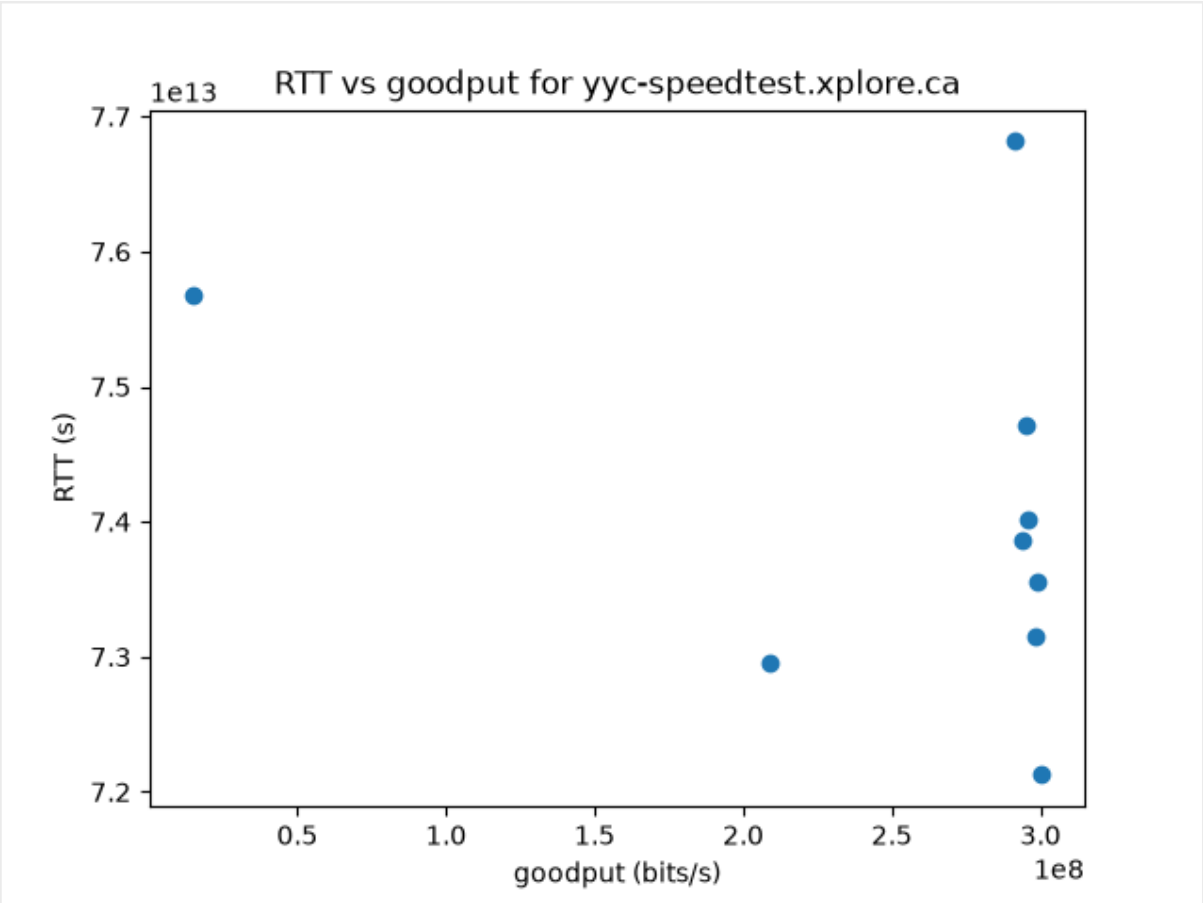
- loss stays stable at 0, as the cubic algorithm remembers the tolerance and doesn't attempt to exceed it



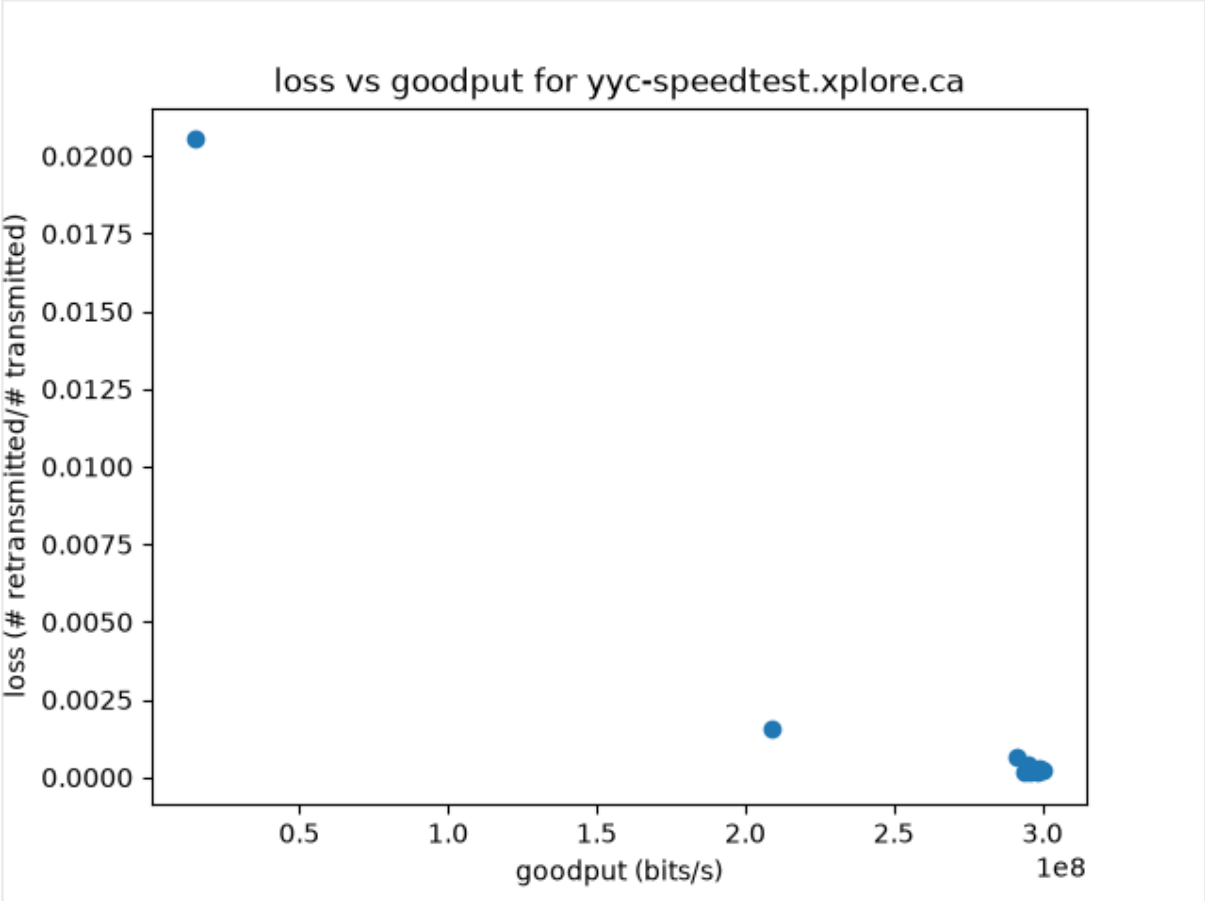
- throughput increases quickly then stabilizes, as the algorithm jumps up using a polynomial function to the remembered tolerance



- this graph is heavily clustered, but shows a roughly linear relation between the send congestion window and the goodput, as a larger window means a larger (expected) possible throughput



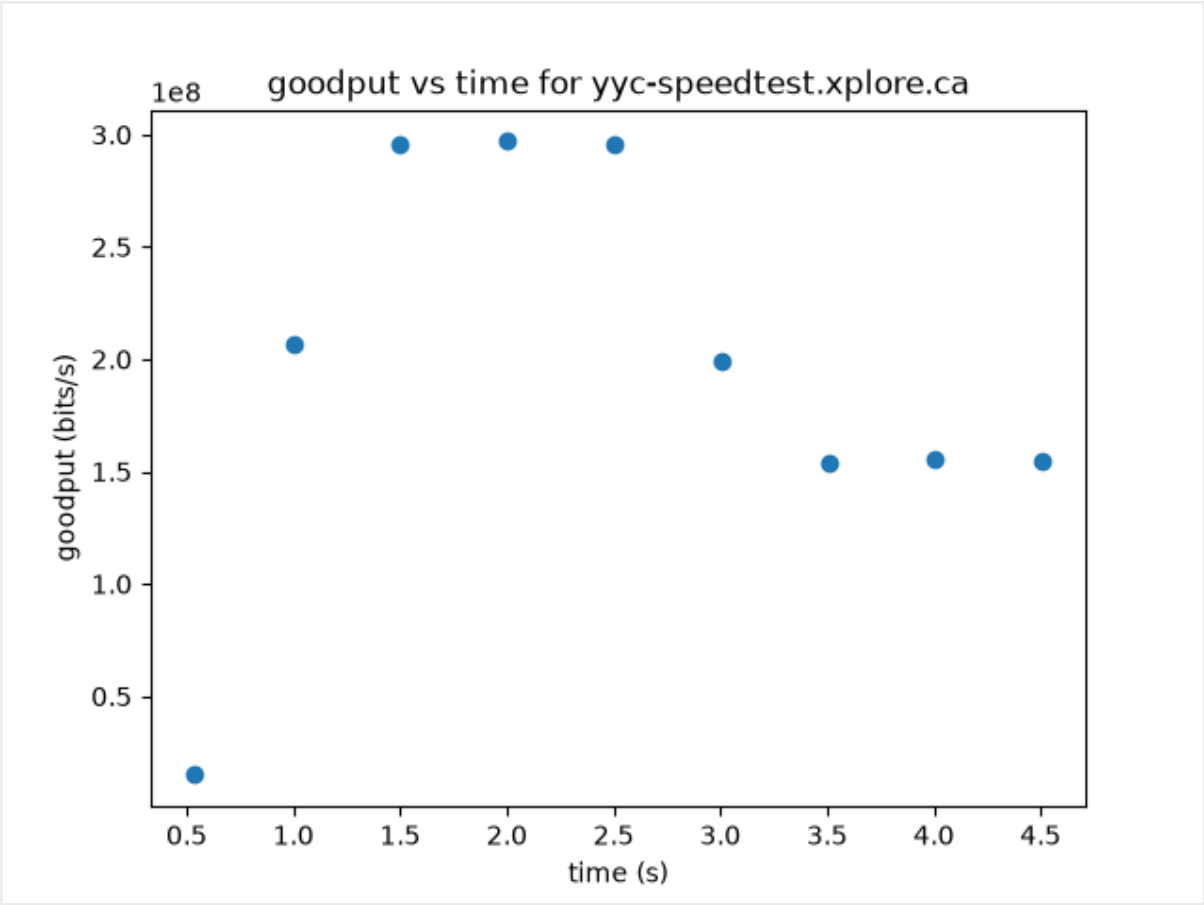
- as before, RTT doesn't have much of a relationship with goodput, it jumps around, irrespective of the throughput

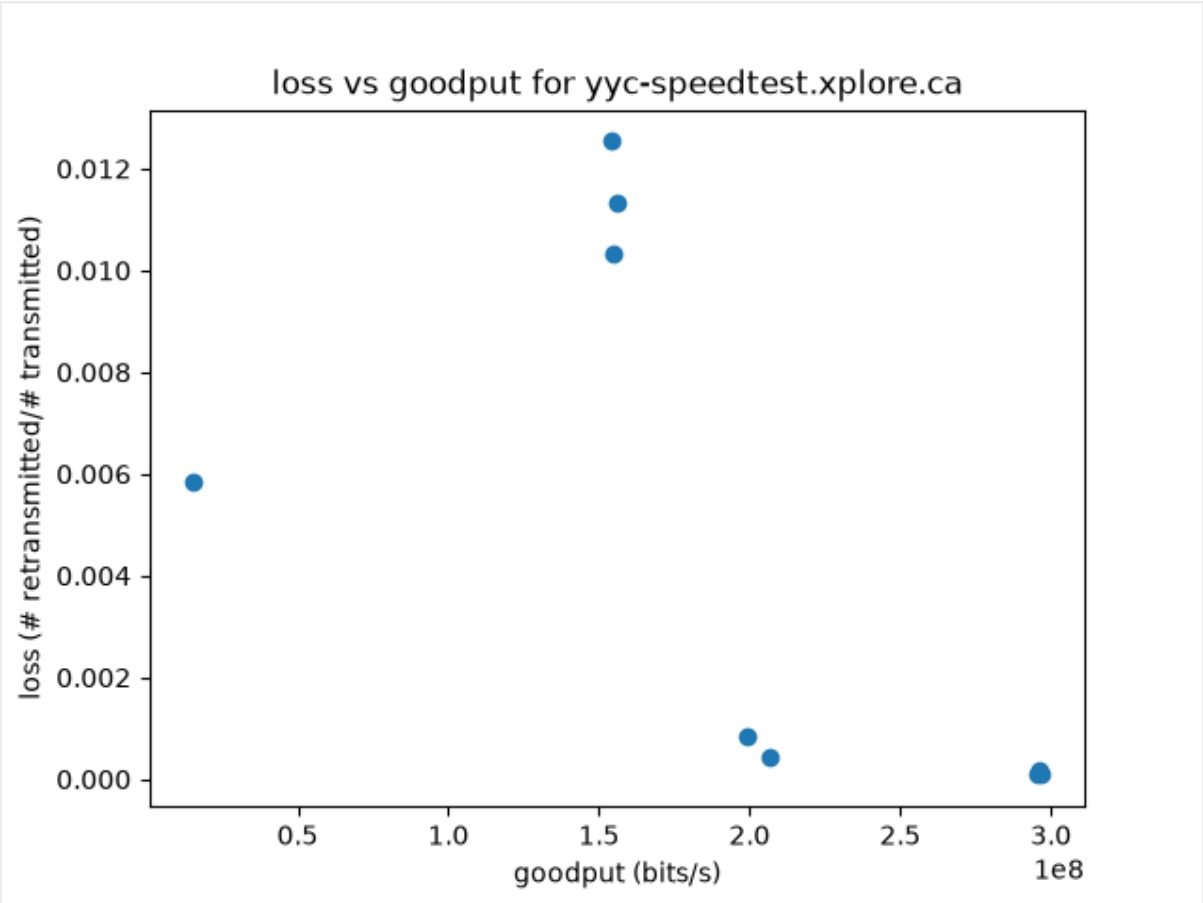


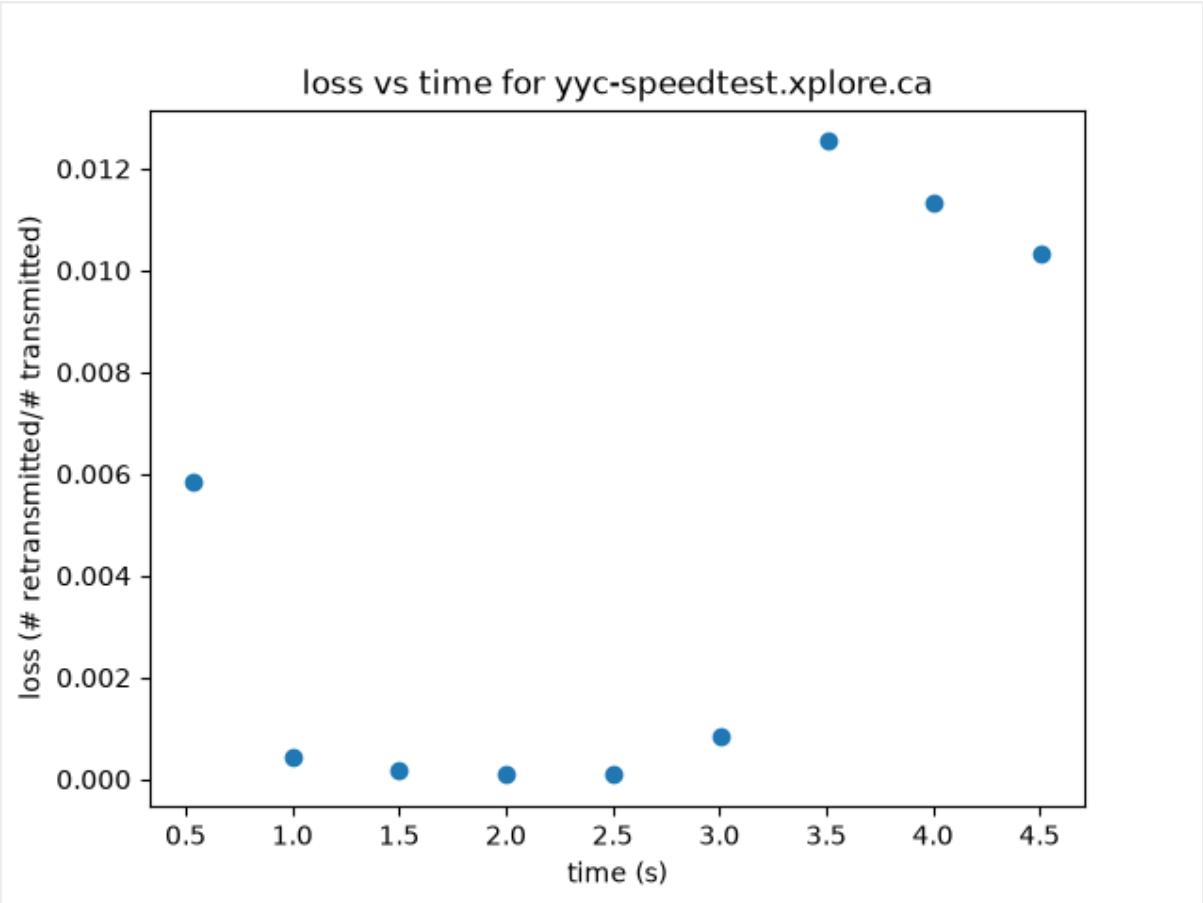
- as expected, loss and goodput are inversely related, with high loss occurring at low goodput and low loss at high goodput

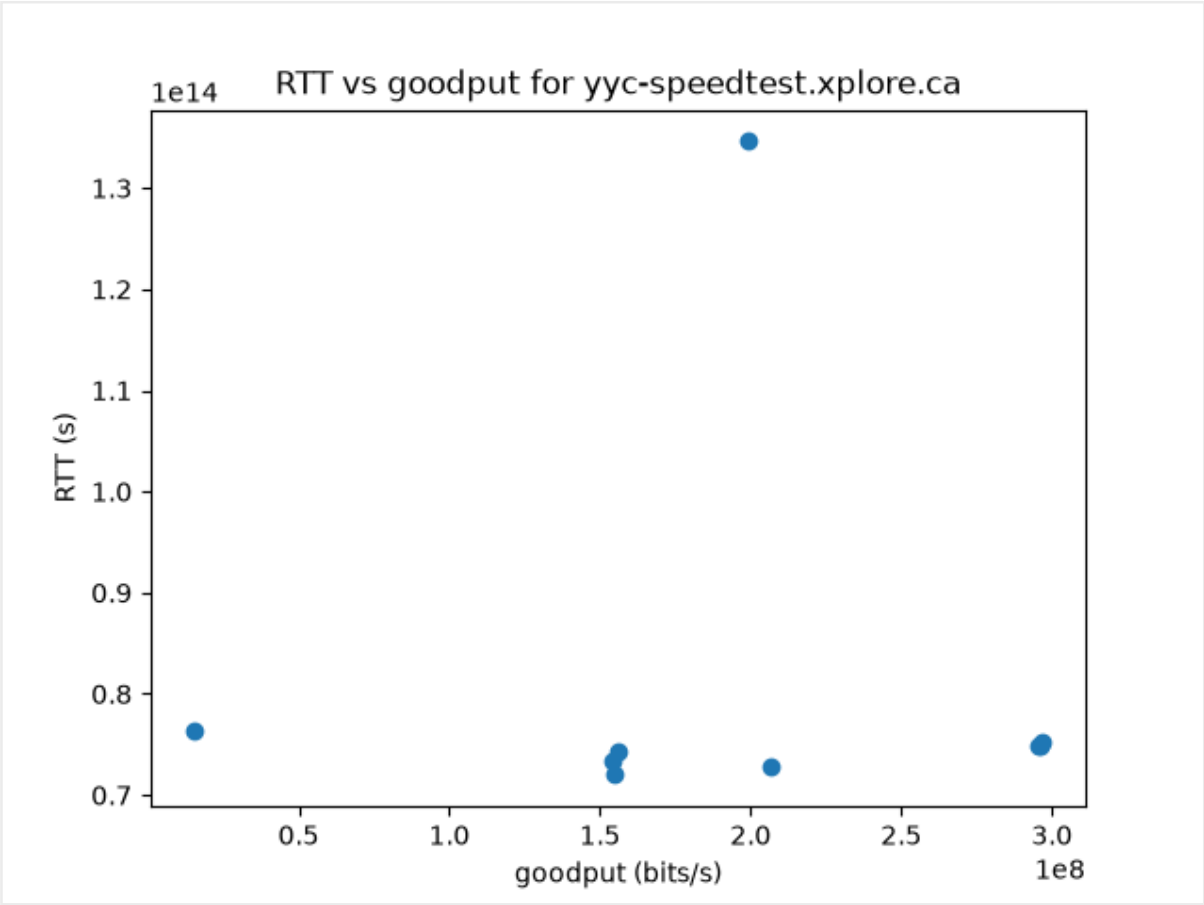
Part 3a

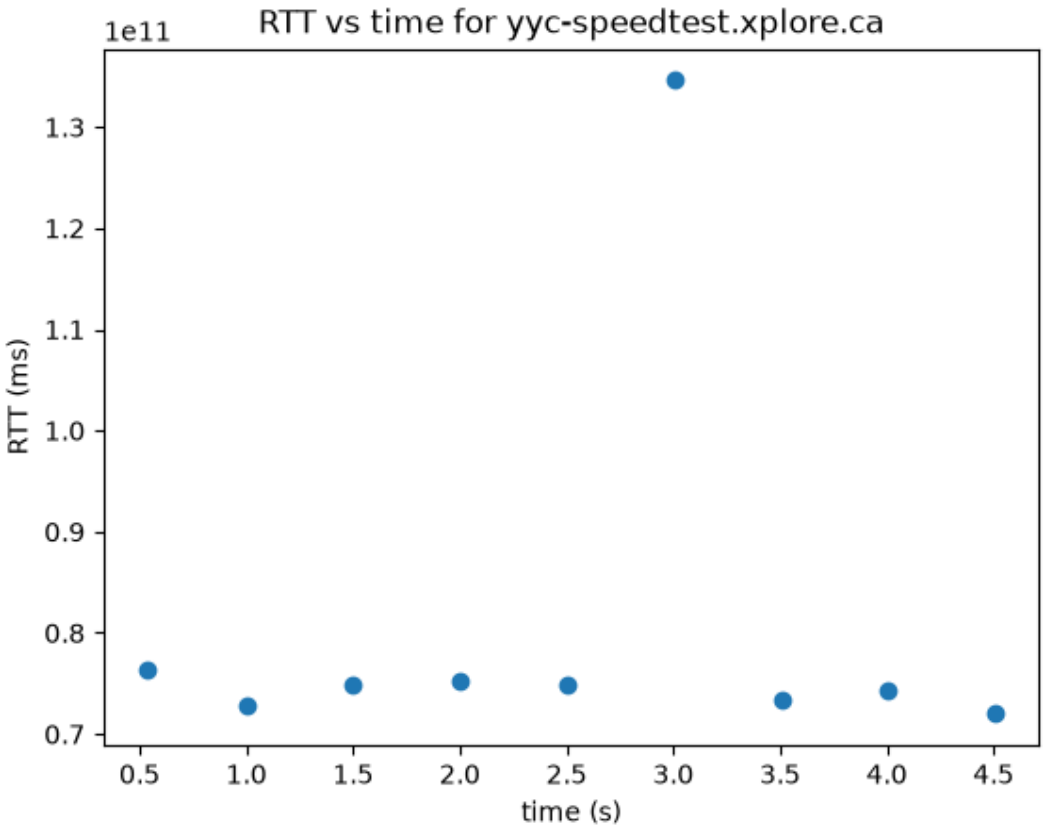
Reno

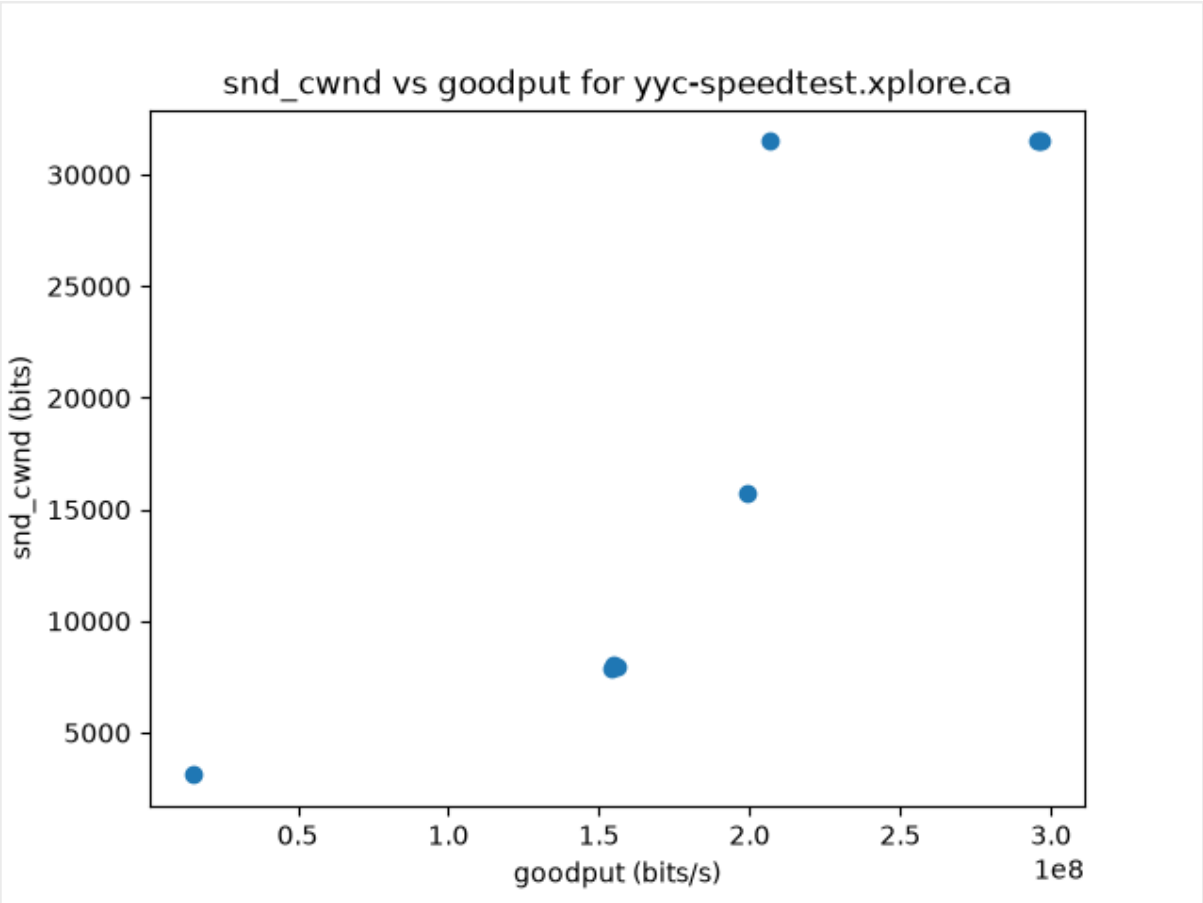

goodput vs time for yyc-speedtest.xplore.ca
1e8
3.0
2.5
2.0
1.5
1.0
0.5
goodput (bits/s)
0.5
1.0
1.5
2.0
2.5
3.0
3.5
4.0
4.5
time (s)

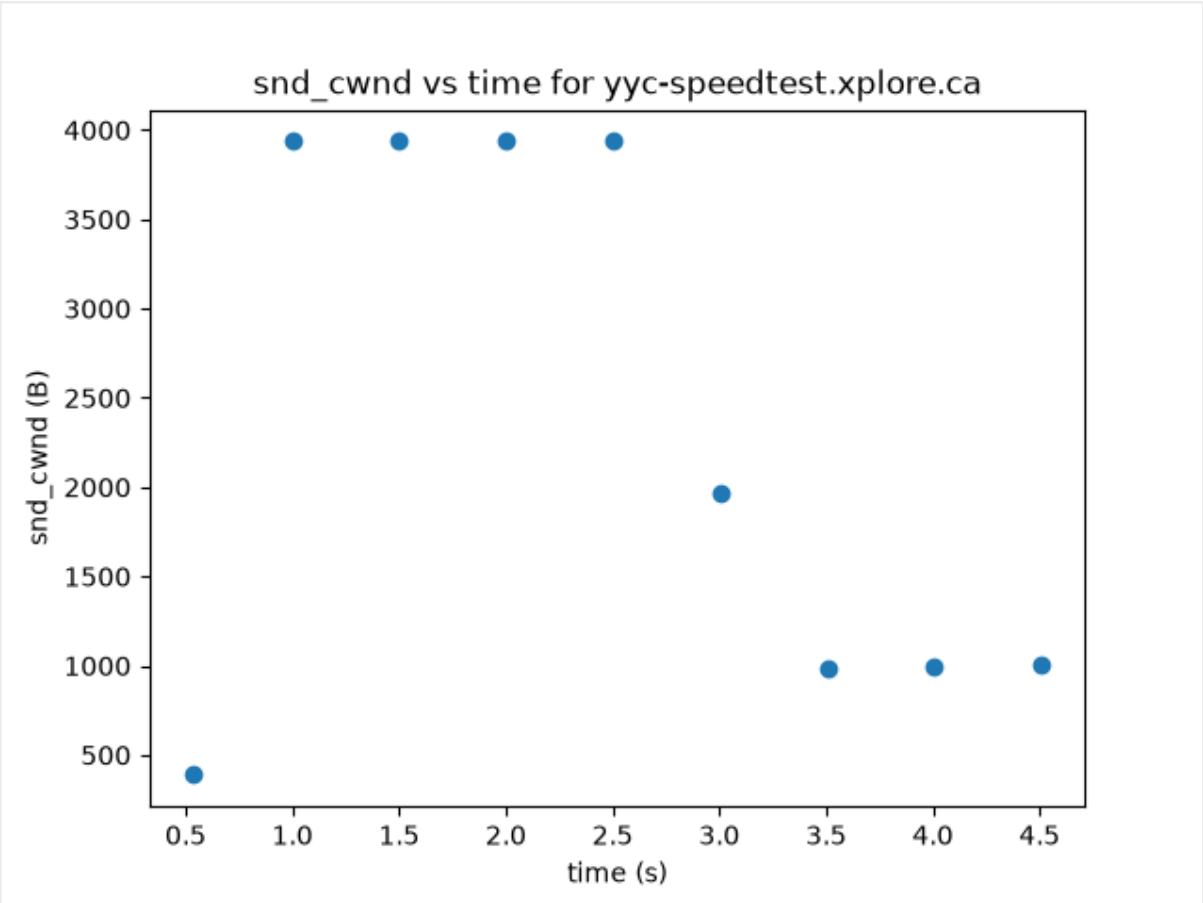

loss vs goodput for yyc-speedtest.xplore.ca
loss (# retransmitted/# transmitted)
0.012
0.010
0.008
0.006
0.004
0.002
0.000
0.5
1.0
1.5
2.0
2.5
3.0
goodput (bits/s)
1e8

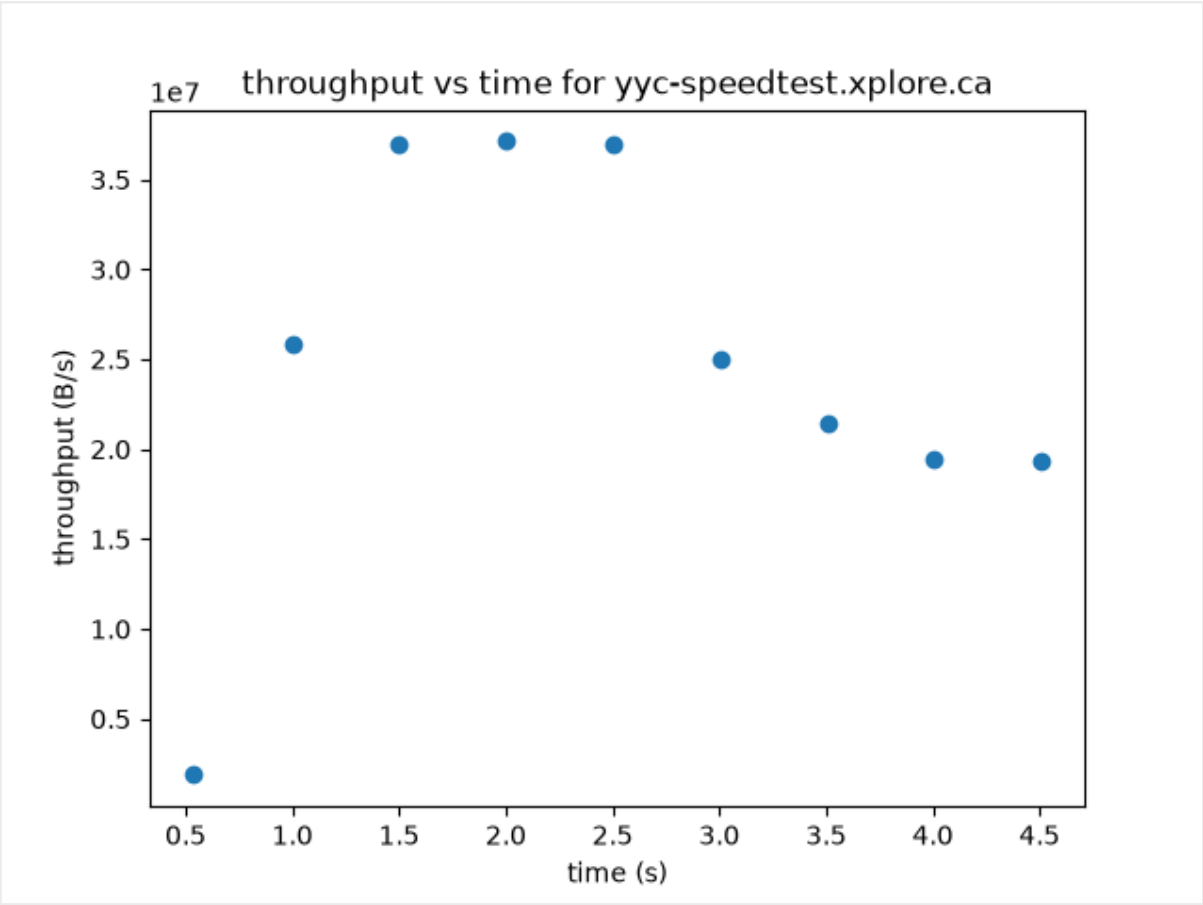

loss vs time for yyc-speedtest.xplore.ca
loss (# retransmitted/# transmitted)
0.012
0.010
0.008
0.006
0.004
0.002
0.000
0.5
1.0
1.5
2.0
2.5
3.0
3.5
4.0
4.5
time (s)


RTT vs goodput for yyc-speedtest.xplore.ca
1e14
1.3
1.2
1.1
1.0
0.9
0.8
0.7
RTT (s)
0.5
1.0
1.5
2.0
2.5
3.0
goodput (bits/s)
1e8

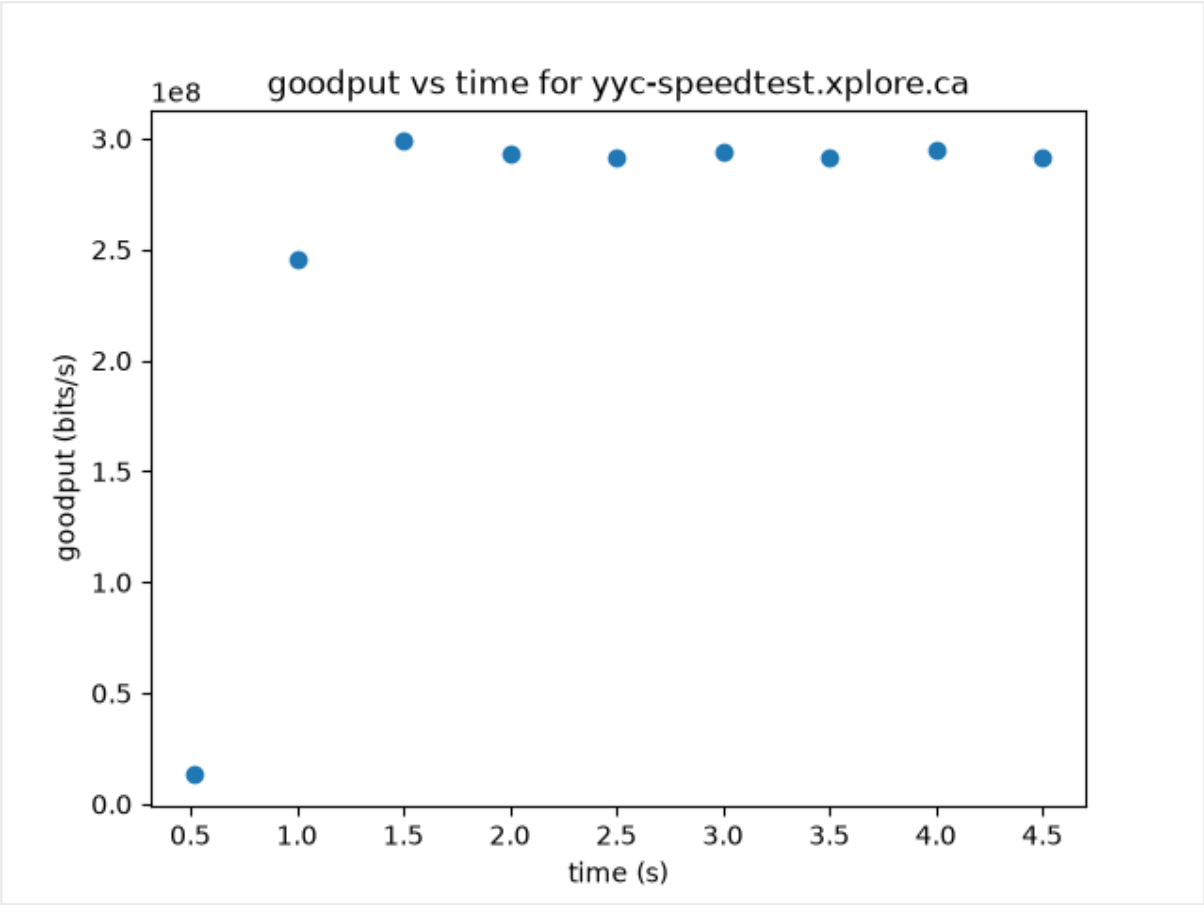

RTT vs time for yyc-speedtest.xplore.ca
1e11
1.3
1.2
1.1
1.0
0.9
0.8
0.7
RTT (ms)
0.5
1.0
1.5
2.0
2.5
3.0
3.5
4.0
4.5
time (s)

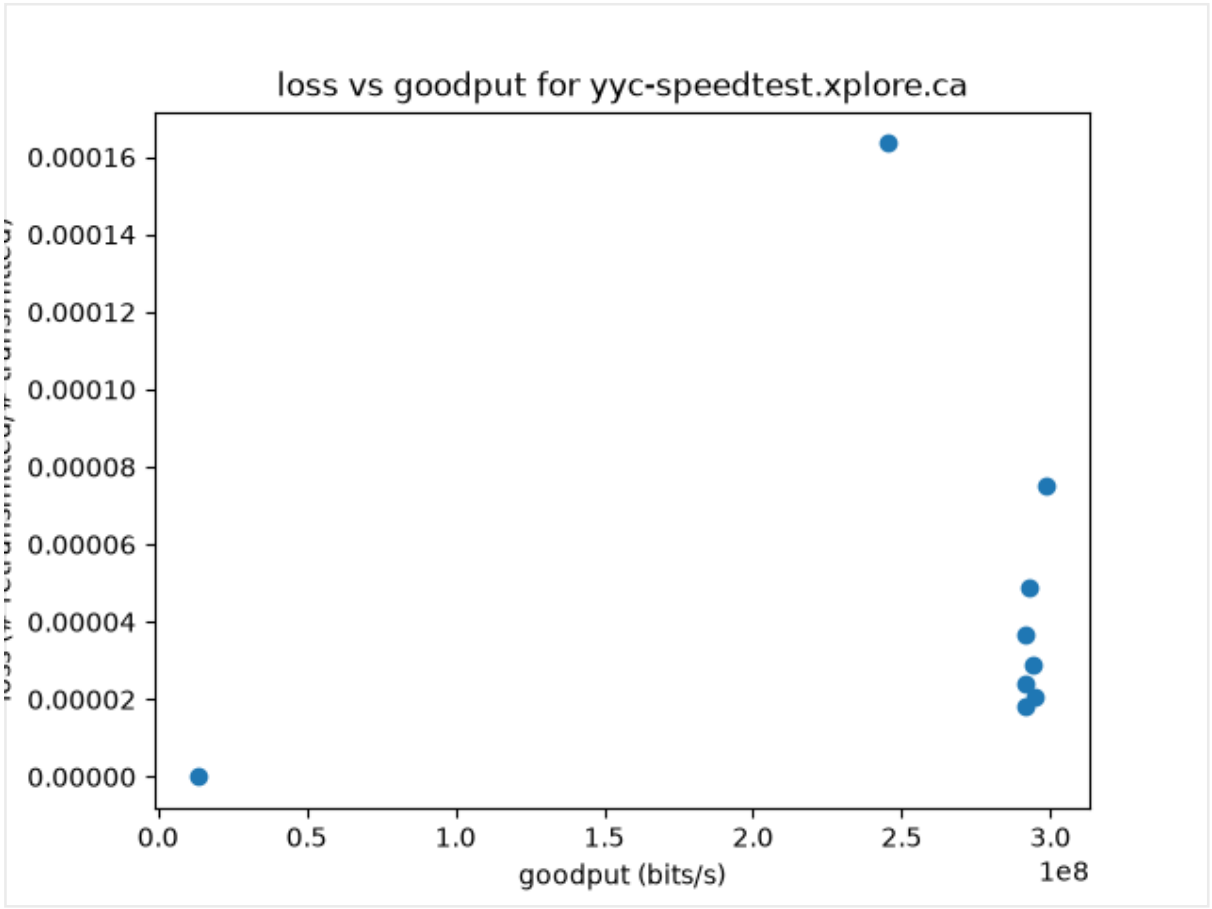

snd_cwnd vs goodput for yyc-speedtest.xplore.ca
snd_cwnd (bits)
30000
25000
20000
15000
10000
5000
0.5
1.0
1.5
2.0
2.5
3.0
goodput (bits/s)
1e8

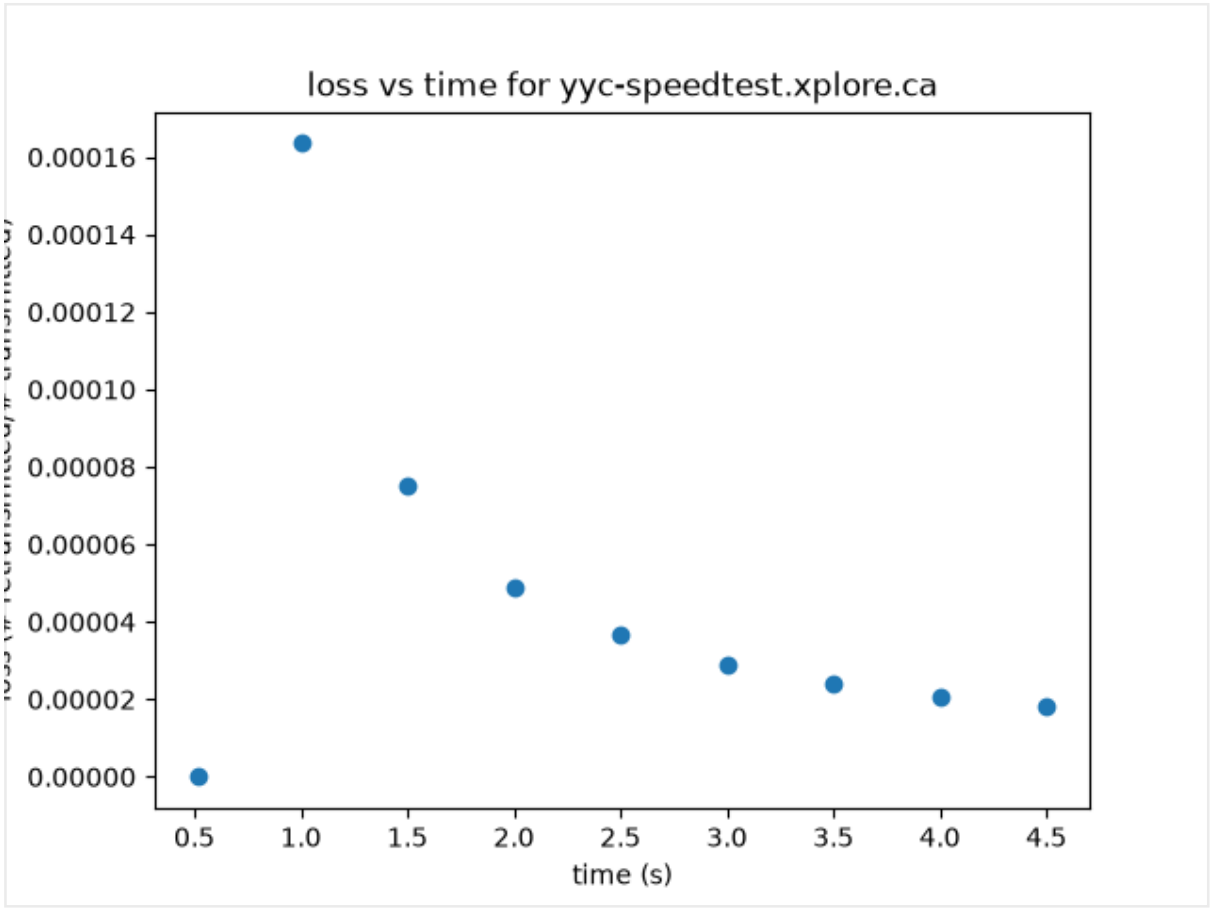

snd_cwnd vs time for yyc-speedtest.xplore.ca
snd_cwnd (B)
4000
3500
3000
2500
2000
1500
1000
500
0.5
1.0
1.5
2.0
2.5
3.0
3.5
4.0
4.5
time (s)

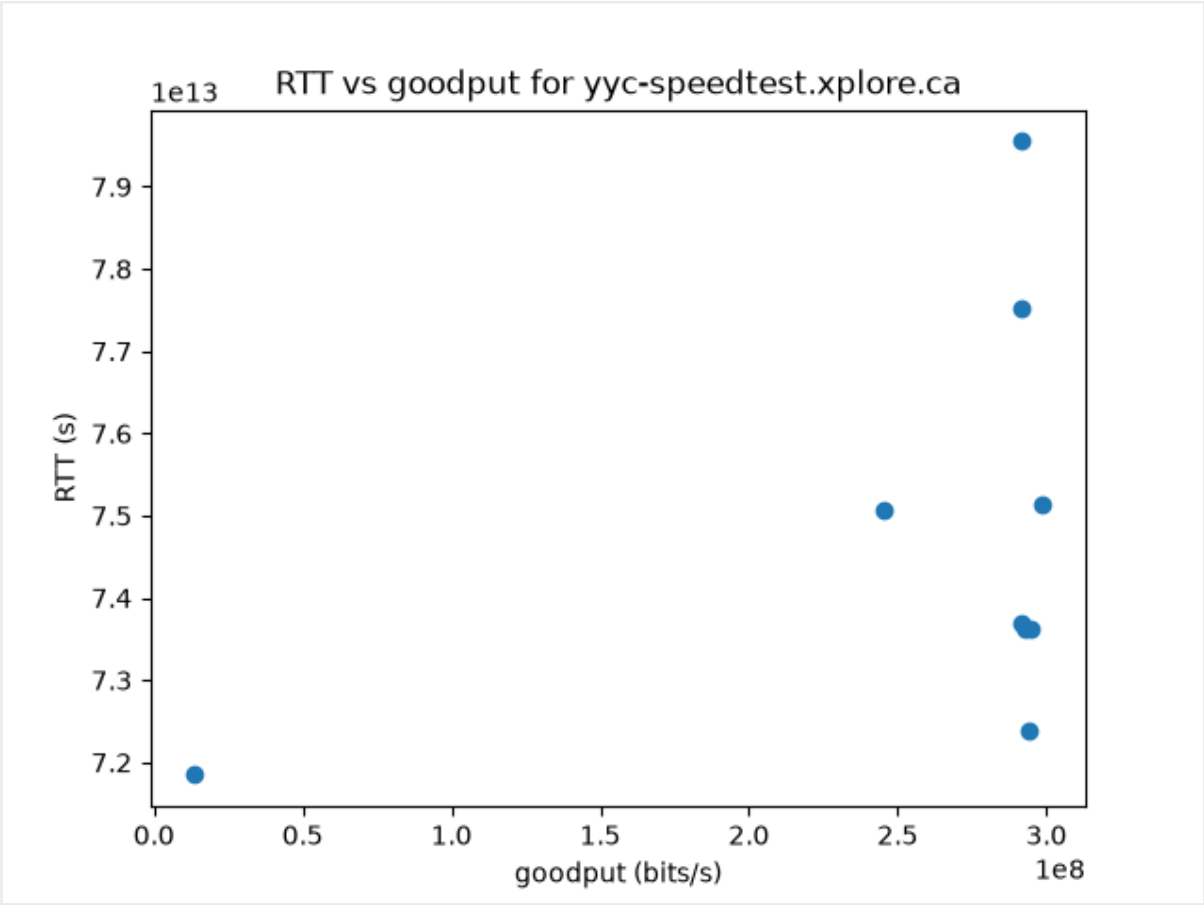

throughput vs time for yyc-speedtest.xplore.ca
1e7
throughput (B/s)
3.5
3.0
2.5
2.0
1.5
1.0
0.5
0.5
1.0
1.5
2.0
2.5
3.0
3.5
4.0
4.5
time (s)

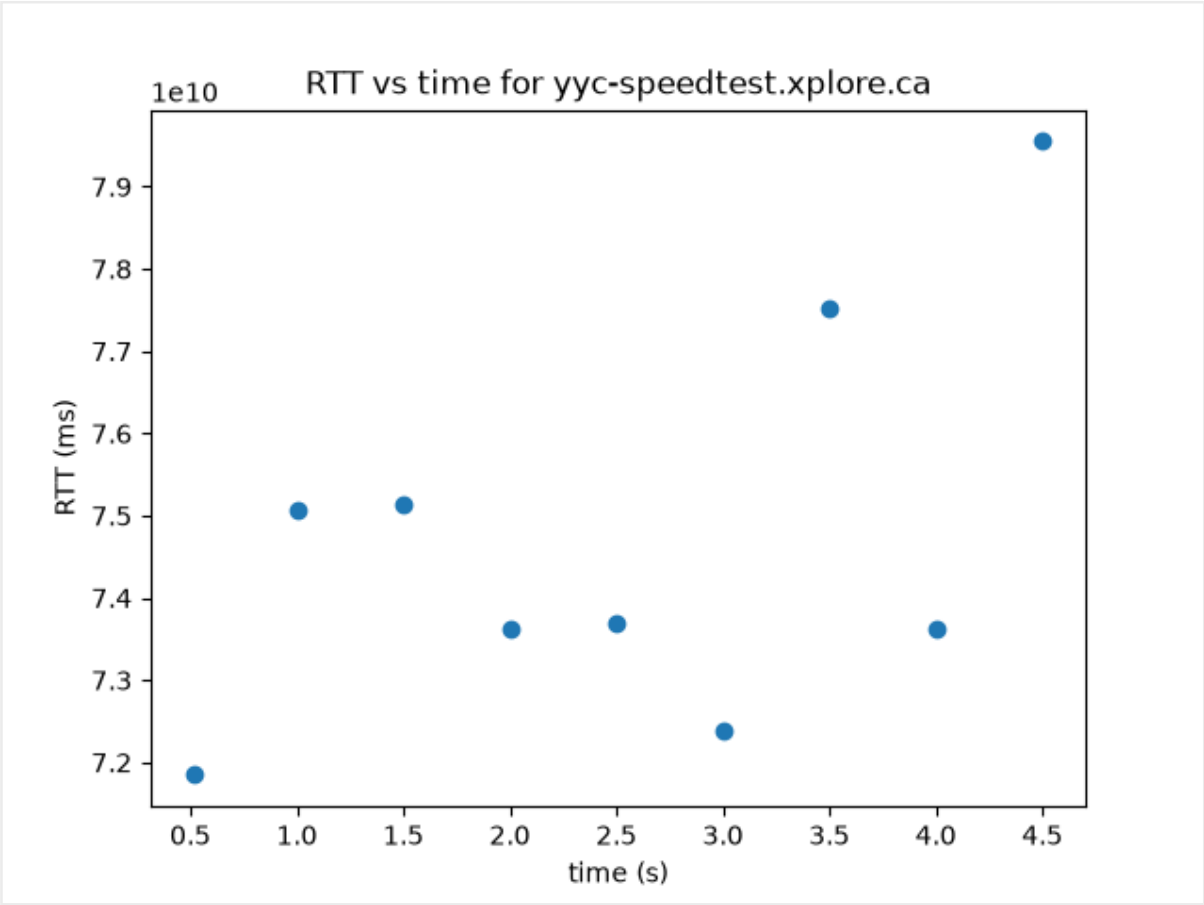
BBR

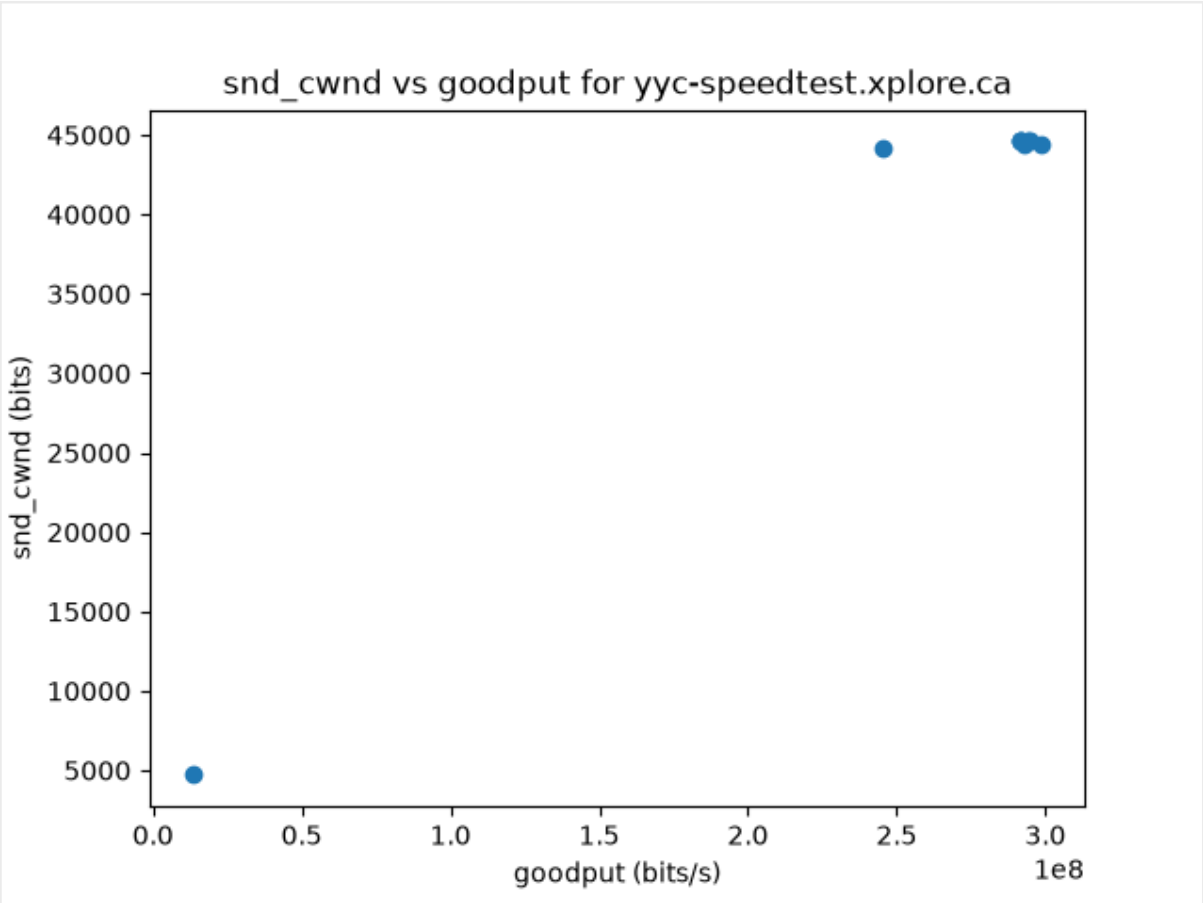

goodput vs time for yyc-speedtest.xplore.ca
1e8
3.0
2.5
2.0
1.5
1.0
0.5
0.0
goodput (bits/s)
0.5
1.0
1.5
2.0
2.5
3.0
3.5
4.0
4.5
time (s)

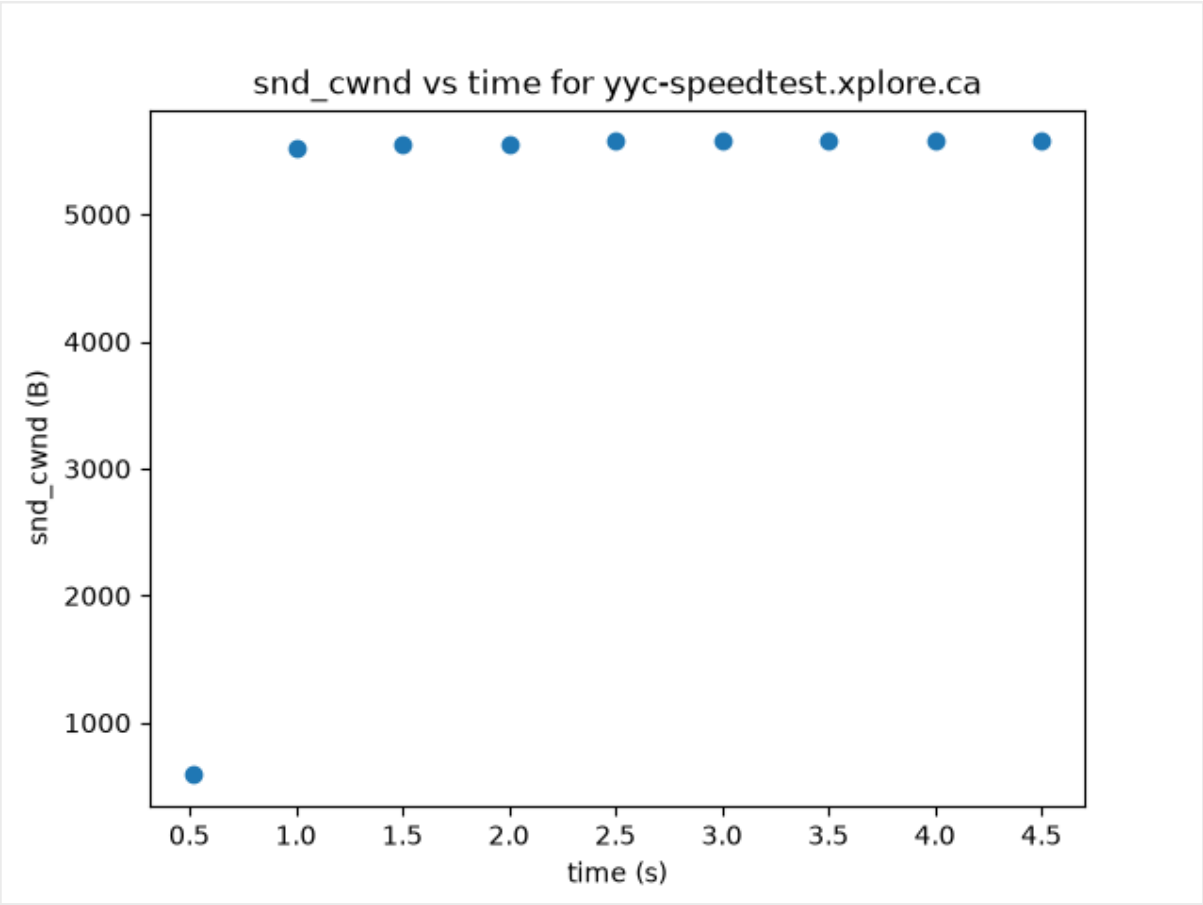

loss vs goodput for yyc-speedtest.xplore.ca
0.00016
0.00014
0.00012
0.00010
0.00008
0.00006
0.00004
0.00002
0.00000
0.0
0.5
1.0
1.5
2.0
2.5
3.0
goodput (bits/s)
1e8

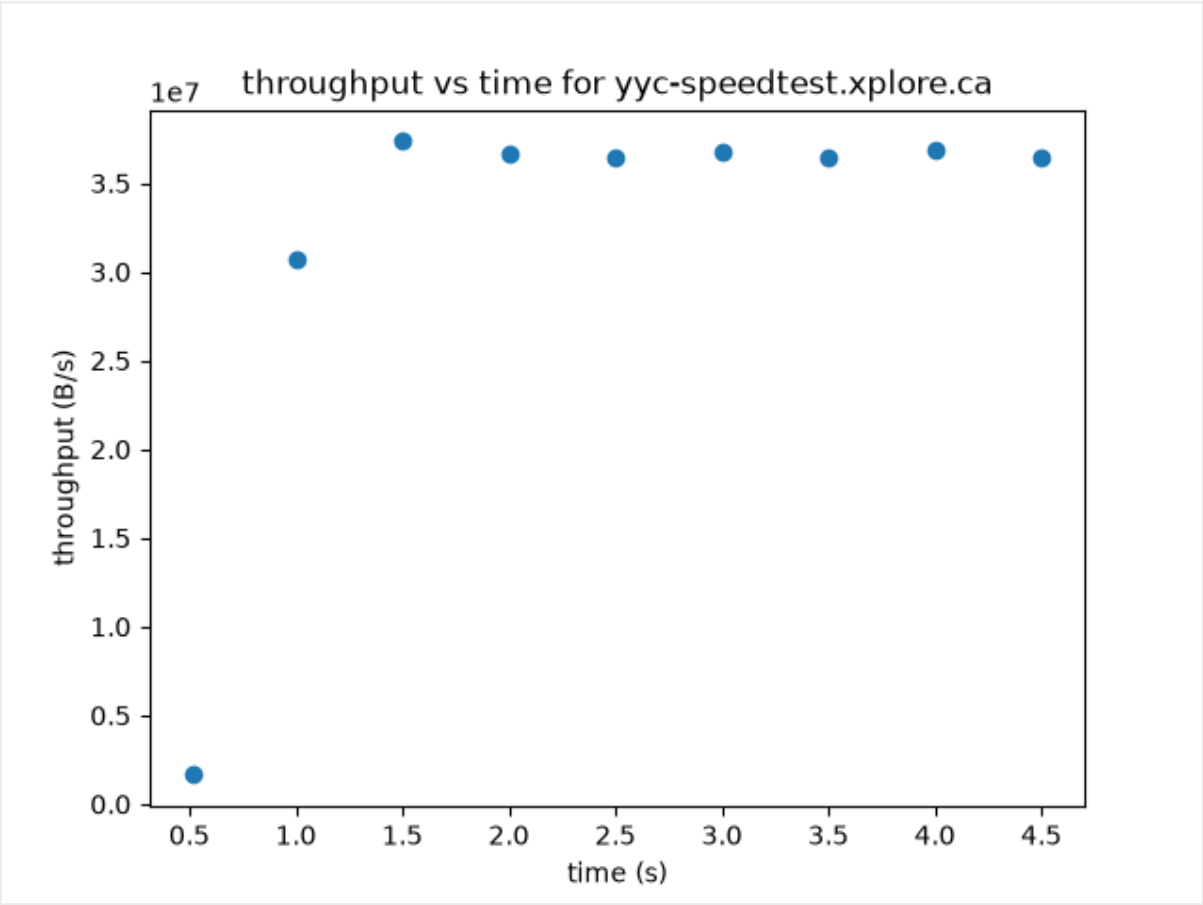

loss vs time for yyc-speedtest.xplore.ca
0.00016
0.00014
0.00012
0.00010
0.00008
0.00006
0.00004
0.00002
0.00000
0.5
1.0
1.5
2.0
2.5
3.0
3.5
4.0
4.5
time (s)


RTT vs goodput for yyc-speedtest.xplore.ca
1e13
7.9
7.8
7.7
7.6
7.5
7.4
7.3
7.2
RTT (s)
0.0
0.5
1.0
1.5
2.0
2.5
3.0
1e8
goodput (bits/s)


RTT vs time for yyc-speedtest.xplore.ca
1e10
7.9
7.8
7.7
7.6
7.5
7.4
7.3
7.2
RTT (ms)
0.5
1.0
1.5
2.0
2.5
3.0
3.5
4.0
4.5
time (s)


snd_cwnd vs goodput for yyc-speedtest.xplore.ca
45000
40000
35000
30000
25000
20000
15000
10000
5000
snd_cwnd (bits)
0.0
0.5
1.0
1.5
2.0
2.5
3.0
1e8
goodput (bits/s)


snd_cwnd vs time for yyc-speedtest.xplore.ca
snd_cwnd (B)
5000
4000
3000
2000
1000
0.5
1.0
1.5
2.0
2.5
3.0
3.5
4.0
4.5
time (s)



STATISTICS

the following measurements were conducted back-to-back with 3 repetitions

```
> python3 main.py -fserverlist.txt 5 0.5 reno 3
testing yyc-speedtest.xplore.ca ... success

summaries for reno:
                   host             minimum
median                mean    95-th percentile
 yyc-speedtest.xplore.ca      15.070 Mbits/s     282.129 Mbits/
s     246.693 Mbits/s     296.605 Mbits/s

> python3 main.py -fserverlist.txt 5 0.5 cubic 3
testing yyc-speedtest.xplore.ca ... success

summaries for cubic:
                   host             minimum
median                mean    95-th percentile
 yyc-speedtest.xplore.ca      13.561 Mbits/s     178.114 Mbits/
s     196.609 Mbits/s     285.563 Mbits/s

python3 main.py -fserverlist.txt 5 0.5 bbr 3
```

```
testing yyc-speedtest.xplore.ca ... success

summaries for bbr:
                     host              minimum
median                  mean    95-th percentile
 yyc-speedtest.xplore.ca      19.805 Mbits/s     150.792 Mbits/
s     123.706 Mbits/s     153.716 Mbits/s
```

BBR was by far the best, followed by CUBIC and then RENO, as expected. In general, when the network is stable BBR should always perform the best, though if the network gets overloaded by other means during a long transmission period, RENO and CUBIC will recover faster (multiplicative decrease), the opposite also being true, if the network suddenly becomes less laden or new paths turn on, RENO and CUBIC will realize the increase faster. The biggest area in which BBR exceeds the other two is it is much more durable against packet loss due to factors other than congestion (queue overflow). BBR also performs best against continuously changing internet paths, as RENO and CUBIC will mistake path (RTT) changes as congestion changes. RENO in particular will get a very wavy pattern which injects a lot of loss.

Part 3b

The congestion control algorithm shall:

- assume that loss is caused by noise rather than congestion and ignore it
- use the RTT and throughput values to make estimates about what the congestion window should be
- start super slow for 1 RTT (near-guaranteed not to cause congestion), then go super fast for 1 RTT (expected to cause congestion)
- now we have obtained the baseline minimum RTT (the first RTT) and bandwidth (calculated from throughput against maximum throughput of the fast period)
- the idea is to make it an asymptote from both sides, or like a sine wave with a decreasing amplitude over time
- the congestion window should converge at the maximum point at which the RTT is equal to the minimum RTT
- even RTTs should be x% below the calculated average, and odd RTTs should be x% above the calculated average, and the calculated average should update on each RTT, x is set to a concrete value for the first two RTTs to create the dramatic difference
- this algorithm will crowd out other congestion algorithms, particularly RENO and CUBIC, but it should play nice with itself, the loud periods will force other users to decrease their rates and let the new player in